

Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«КУБАНСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ»
(ФГБОУ ВО «КубГУ»)



МЕТОДИЧЕСКИЕ УКАЗАНИЯ

**по выполнению лабораторных работ по дисциплине
«Технологии проектирования программного обеспечения»**

для бакалавров направлений подготовки 01.03.02 «Прикладная математика и информатика», 09.03.03 «Прикладная информатика» профиль «Прикладная информатика в экономике»

Краснодар,
2024

Методические указания к выполнению лабораторных работ по дисциплине «Технологии проектирования программного обеспечения», для бакалавров направлений подготовки 01.03.02 «Прикладная математика и информатика», 09.03.03 «Прикладная информатика» профиль «Прикладная информатика в экономике» / Составители: доц. каф. ИТ Полетайкин А.Н., доц. каф. ИТ Добровольская Н.Ю. – Краснодар: КубГУ, 2024. – 77 с.

Методические указания содержат теоретические положения, задания и методические указания к выполнению лабораторных работ по дисциплине «Технологии проектирования программного обеспечения».

Составители: доц. каф. ИТ, к.т.н., доц. Полетайкин А.Н.

доц. каф. ИТ, к.пед.н., доц. Добровольская Н.Ю.

СОДЕРЖАНИЕ

1	Общие положения	5
1.1	Организация занятий по курсу.....	5
1.2	Требования к содержанию отчетов о выполнении лабораторных работ.....	7
1.3	Требования к оформлению отчетной документации	7
2	Методические указания	10
	Лабораторная работа №1	10
	Теоретические положения	10
	1.1. Принцип системного анализа	10
	1.2. Системный подход к описанию объекта и процесса информатизации.....	10
	1.3. Модель вариантов использования.....	14
	Задание	16
	Требования к содержанию отчета.....	16
	Контрольные вопросы и упражнения.....	18
	Лабораторная работа №2	19
	Задание	19
	Требования к содержанию отчета.....	19
	Контрольные вопросы	20
	Лабораторная работа №3	21
	Задание	21
	Требования к содержанию отчета.....	21
	Контрольные вопросы и упражнения.....	24
	Лабораторная работа №4	26
	Теоретические положения	26
	4.1 Функционально-ориентированный подход к проектированию ПС	26
	4.2. Пример функционального моделирования ПС.....	28
	Задание	29
	Требования к содержанию отчета.....	30
	Контрольные вопросы и упражнения.....	30
	Лабораторная работа №5	32
	Задание	32
	Требования к содержанию раздела.....	34
	Контрольные вопросы	35
	Лабораторная работа №6	36
	Теоретические положения	36
	6.1. CASE-средство ERwin для моделирования баз данных.....	36
	6.2. Разработка логической модели данных	37
	6.3. Разработка физической модели данных.....	40
	Задание	43
	Требования к содержанию раздела.....	43
	Контрольные вопросы и упражнения.....	44
	Лабораторная работа №7	45
	Теоретические положения	45
	7.1. Классификация ПО	45

7.2. Диаграммы компонентов UML	46
7.3. Диаграммы развертывания UML.....	47
Задание	49
Требования к содержанию отчета.....	50
Контрольные вопросы и упражнения.....	51
Лабораторная работа №8	52
Теоретические положения	52
8.1. Тестирование, как способ контроля качества ПО.	52
8.2. Критерии тестирования.....	52
8.3. Виды тестирования	53
8.4. Сценарное тестирование	54
8.5. Интеграционное тестирование	55
Задание	55
Требования к содержанию отчета.....	56
Контрольные вопросы и упражнения.....	57
Лабораторная работа №9	58
Теоретические положения	58
9.1. Структура баг-репорта	58
9.2. Основные характеристики и виды дефектов тестируемого ПО	59
9.3. Требования к количеству открытых дефектов	59
9.4. Требования к обязательным полям баг-репорта.....	60
9.5. Основные ошибки при написании баг-репортов	61
Задание	62
Контрольные вопросы и упражнения.....	62
Лабораторная работа №10.....	63
Теоретические положения	63
10.1. Документирование ПО.....	63
10.2. Руководство пользователя	63
Задание	64
Контрольные вопросы и упражнения.....	64
Список рекомендуемой литературы	65
Приложение А – Варианты индивидуальных заданий	67
Приложение Б – Примерное описание объекта информатизации.....	70
Приложение В – Показатели для оценивания программной системы (согласно ГОСТ Р ИСО/МЭК 9126-93).....	73
Приложение Г – Состав и содержание технического задания на создание программ (ГОСТ 34.602- 89)	75
Приложение Д – Дополнительное задание на проведение сценарного тестирования.	76

1 Общие положения

Дисциплина «Технологии проектирования программного обеспечения» рассматривает методы и инструментальные средства программной инженерии для решения задач создания качественного программного обеспечения в разных сферах деятельности человека с применением современных информационных и компьютерных технологий.

Изучаются основные понятия, методы и модели программной инженерии, составляющие процесса разработки программного обеспечения, управление требованиями к программной системе, конфигурационное управление программным продуктом, методы, способы и порядок тестирования программного обеспечения, управление версиями и сборками. Рассматриваются и применяются на практике методы, способы и инструментальные средства для анализа предметных областей, постановки задачи, формулирования требований к программе, компилятивной сборки и тестирования специального программного обеспечения для экономики и современного бизнеса.

1.1 Организация занятий по курсу

Дисциплина «Технологии проектирования программного обеспечения» читается студентам 02.03.03 «Математическое обеспечение и администрирование информационных систем» профиль «Технология программирования» и 09.03.03 «Прикладная информатика» профиль «Прикладная информатика в экономике» на третьем курсе обучения, а студентам 01.03.02 «Прикладная математика и информатика» на четвертом курсе обучения.

Целью курса является формирование у студентов знаний, умений и практических навыков в области анализа и системного представления бизнес-процессов, разработки технического задания на создание специального программного обеспечения для решения поставленных задач, организации программного процесса на всех стадиях жизненного цикла программного обеспечения (ПО).

Предметом учебной дисциплины являются методы, подходы и инструментальные средства программной инженерии и технологии проектирования ПО.

Задачами дисциплины является получение представления о жизненном цикле многоверсионного программного продукта, а также приобретения навыков применения знаний и умений, приобретенных в бакалавриате, для создания сложных программных проектов, отвечающих требованиям современного бизнеса.

Изучаемый курс предусматривает выполнение 9 лабораторных работ и предполагает создание программной системы (ПС) и её специального программного обеспечения для реализации обработки информации на ЭВМ в рамках заданного бизнес-процесса ([см. приложение А](#)). Рабочая схема выполнения лабораторных занятий представлена в табл. 1.

Таблица 1. Рабочая схема выполнения лабораторных работ

№ л. р.	Тема лабораторной работы: материал для изучения и характер выполняемых работ	Неделя выполнения
1.	<u>Анализ предметной области</u> : выбор и утверждение индивидуальной темы, системный анализ объекта и процесса информатизации, характеристика решения задач и выделение ее недостатков, обоснование необходимости усовершенствования существующего решения задач	1 – 2
2.	<u>Анализ существующих компьютерных разработок</u> : системное описание существующих подобных ПС, сравнительная характеристика описанных систем	3
3.	<u>Техническое задание на создание ПС</u> : назначение и общая цель создания программы, структура программы и состав функциональных задач, функциональные и нефункциональные требования к программе, моделирование требований на языке UML	4 – 5
4.	<u>Проектирование функциональной структуры ПС</u> : построение и документирование функциональной модели разрабатываемого ПО в виде контекстной диаграммы и ее декомпозиции в нотации IDEF0	6 – 7
5.	<u>Объектно-ориентированный подход к проектированию</u> : Проектирование функциональной структуры разрабатываемого ПО в нотации UML	8 – 9
6.	<u>Проектирование базы данных ПС</u> : изучение программных средств для разработки моделей информационной базы ПС, проработка методов нормализации отношений в БД, создание БД ПС при помощи выбранной СУБД	10 – 11
7.	<u>Разработка программного обеспечения ПС</u> : технология программирования прикладных задач, разработка интерфейсной части ПС при помощи современных средств разработки, построение диаграмм UML физической реализации разработанной ПС, анализ производительности программы, оценивание эффективности кода	12 – 13
8.	<u>Тестирование ПС</u> : проверка работоспособности программы, сценарное, регрессионное, нагрузочное, пользовательское тестирование	14 – 15
9.	<u>Разработка баг-репорта ПС</u> : поиск и классификация дефектов программного продукта, составление баг-репортов	16
10.	<u>Документирование и развертывание ПС</u> : освоение методики документирования ПС, разработка функциональной спецификации ПС и руководства пользователя	17

По каждой лабораторной работе оформляется отчет. Отчеты сдаются на проверку руководителю в течение курса по мере их выполнения, и защищаются студентами в установленном порядке.

При защите отчета студенту могут быть заданы вопросы и дополнительные задания по сути лабораторной работы, в том числе из списка контрольных вопросов к данной лабораторной работе. При неудовлетворительной оценке знаний студента по теме данного отчета, студент возвращается к повторному изучению соответствующих материалов, после чего допускается к повторной защите. Неудовлетворительно выполненный отчет также возвращается на доработку. Требования к содержанию отчетов о выполнении лабораторных работ представлены в [разделе 1.2](#).

1.2 Требования к содержанию отчетов о выполнении лабораторных работ

В процессе изучения курса «Технологии проектирования программного обеспечения» должны быть подготовлены и сданы десять отчетов о выполнении лабораторных работ (см. [таблицу 1](#)). Лабораторные работы выполняются в соответствии с заданием (раздел 2) в рамках индивидуальной темы.

Отчет должен содержать заголовок, тему лабораторной работы, цель, задание, индивидуальную тему, описание хода выполнения работы, необходимые прикладные материалы (схемы, макеты документов и т.п.), в соответствии с требованиями к содержанию, представленными в соответствующем подразделе раздела 2, и выводы по работе.

1.3 Требования к оформлению отчетной документации

Отчет должен быть выполнен печатным способом с использованием компьютера и принтера на одной стороне листа белой бумаги формата А4 через полтора интервала. Для создания текста работы рекомендуется использовать текстовый редактор MS Word. Шрифт на протяжении всего документа должен быть одинаковый: Times New Roman 14-го размера, за исключением оформления иллюстраций, таблиц и формул, в которых допускается использовать шрифт меньшего размера. Минимально допустимый размер шрифта – 12.

Текст отчета следует печатать, соблюдая следующие размеры полей: слева – 25-30 мм, справа – 10 мм, сверху и снизу – 20 мм.

При оформлении текста работы следует использовать абзацный отступ, который должен составлять 15-17 мм от левого поля документа

Вне зависимости от способа выполнения отчета качество напечатанного текста и оформления иллюстраций, таблиц, распечаток с ПЭВМ должно удовлетворять требованию их четкого воспроизведения.

Нумерация страниц отчета. Страницы отчета следует нумеровать арабскими цифрами, соблюдая сквозную нумерацию по всему тексту отчета. Номер страницы проставляют в центре нижней части листа без точки. Титульный лист включают в общую нумерацию страниц отчета. Номер страницы на титульном листе не проставляют.

Иллюстрации и таблицы, расположенные на отдельных листах, включают в общую нумерацию страниц отчета.

Иллюстрации. Иллюстрации (чертежи, графики, схемы, компьютерные распечатки, диаграммы, фотоснимки) следует располагать в отчете непосредственно после текста, в котором они упоминаются впервые, или на следующей странице. Иллюстрации могут быть в компьютерном исполнении, в том числе и цветные. На все иллюстрации должны быть даны ссылки в отчете.

Иллюстрации следует нумеровать арабскими цифрами сквозной нумерацией. Если рисунок один, то он обозначается “Рисунок 1”. Слово “Рисунок” и его наименование располагают посередине строки. Допускается нумеровать иллюстрации в пределах раздела. В этом случае номер иллюстрации состоит из номера раздела и порядкового номера иллюстрации, разделенных точкой. Например, Рисунок 1.1. Иллюстрации, при необходимости, могут иметь наименование и пояснительные данные (подрисовочный текст). Слово “Рисунок” и наименование помещают после пояснительных данных и располагают следующим образом: Рисунок 1.1 — Функциональная схема.

При ссылках на иллюстрации следует писать “... в соответствии с рисунком 2” при сквозной нумерации и “... в соответствии с рисунком 1.2” при нумерации в пределах раздела.

Таблицы применяют для лучшей наглядности и удобства сравнения показателей. Название таблицы, при его наличии, должно отражать ее содержание, быть точным, кратким. Название таблицы следует помещать над таблицей слева, без абзацного отступа в одну строку с ее номером через тире.

При переносе части таблицы название помещают только над первой частью таблицы, нижнюю горизонтальную черту, ограничивающую таблицу, не проводят.

Таблицу следует располагать в отчете непосредственно после текста, в котором она упоминается впервые, или на следующей странице. На все таблицы должны быть ссылки в отчете. При ссылке следует писать слово “таблица” с указанием ее номера.

Таблицу с большим количеством строк допускается переносить на другой лист (страницу). При переносе части таблицы на другой лист (страницу) слово “Таблица” и номер ее указывают один раз справа над первой частью таблицы, над другими частями пишут слово “Продолжение” и указывают номер таблицы, например: “Продолжение таблицы 1”. При переносе таблицы на другой лист (страницу) заголовок помещают только над ее первой частью.

Таблицу с большим количеством граф допускается делить на части и помещать одну часть под другой в пределах одной страницы. Если строки и графы таблицы выходят за формат страницы, то в первом случае в каждой части таблицы повторяется головка, во втором случае — боковик.

Если повторяющийся в разных строках графы таблицы текст состоит из одного слова, то его после первого написания допускается заменять кавычками; если из двух и более слов, то при первом повторении его заменяют словами “То же”, а далее — кавычками. Ставить кавычки вместо повторяющихся цифр, марок, знаков, математических и химических символов не допускается. Если цифровые или иные данные в какой-либо строке таблицы не приводят, то в ней ставят прочерк.

Таблицы следует нумеровать арабскими цифрами сквозной нумерацией. Допускается нумеровать таблицы в пределах раздела. В этом случае номер таблицы состоит из номера раздела и порядкового номера таблицы, разделенных точкой.

Заголовки граф и строк таблицы следует писать с прописной буквы в единственном числе, а подзаголовки граф – со строчной буквы, если они составляют одно предложение с заголовком, или с прописной буквы, если они имеют самостоятельное значение. В конце заголовков и подзаголовков таблиц точки не ставят. Таблицы слева, справа и снизу, как правило, ограничивают линиями. Допускается применять размер шрифта в таблице меньший, чем в тексте. Горизонтальные и вертикальные линии, разграничивающие строки таблицы, допускается не проводить, если их отсутствие не затрудняет пользование таблицей. Заголовки граф, как правило, записывают параллельно строкам таблицы. При необходимости допускается перпендикулярное расположение заголовков граф. Головка таблицы должна быть отделена линией от остальной части таблицы.

Формулы и уравнения. Уравнения и формулы следует выделять из текста в отдельную строку. Выше и ниже каждой формулы или уравнения должно быть оставлено не менее одной свободной строки. Если уравнение не умещается в одну строку, то оно должно быть перенесено после знака равенства (=) или после знаков плюс (+), минус (-), умножения (x), деления (:), или других математических знаков, причем знак в начале следующей строки повторяют. При переносе формулы на знаке, символизирующем операцию умножения, применяют знак “X”.

Пояснение значений символов и числовых коэффициентов следует приводить непосредственно под формулой в той же последовательности, в которой они даны в формуле.

Формулы в отчете следует нумеровать порядковой нумерацией в пределах всего отчета арабскими цифрами в круглых скобках в крайнем правом положении на строке. Одну формулу обозначают – (1). Ссылки в тексте на порядковые номера формул дают в скобках. Пример – ... в формуле (1). Допускается нумерация формул в пределах раздела. В этом случае номер формулы состоит из номера раздела и порядкового номера формулы, разделенных точкой, например (3.1).

Ссылки. При ссылках на стандарты и технические условия указывают только их обозначение, при этом допускается не указывать год их утверждения при условии полного описания стандарта в списке использованных источников в соответствии с ГОСТ 7.1. Ссылки на использованные источники следует приводить в квадратных скобках.

2 Методические указания

Лабораторная работа №1

Тема: Анализ предметной области.

Цель: изучение и системное представление бизнес-процессов, подлежащих программированию, приобретение навыков системного анализа объектов и процессов реального мира на предмет организации программного управления ими.

Теоретические положения

1.1. Принцип системного анализа

При системном анализе определяется целевая функция – результат работы изучаемой системы (например, целевая функция работы фабрики – производство продукции, парикмахерская – выполнение стрижек и причесок, и т. д.).

Сущность системного анализа заключается в том, что система разделяется на ряд подсистем (частей), а каждая подсистема в свою очередь делится на задачи.

Понятие «подсистема» подразумевает, что выделяется относительно независимая часть системы, обладающая свойствами системы и, в частности, имеющая подцель, на достижение которой ориентирована подсистема, а также другие свойства – целостности, коммуникативности и т.п., определяемые закономерностями систем.

Система (процесс) может быть разделена на элементы (задачи) не сразу, а последовательным расчленением на подсистемы – совокупности элементов. Такое расчленение, как правило, производится на основе определения независимой функции, выполняемой данной совокупностью элементов совместно для достижения некой частной цели, которая обеспечивает достижение общей цели системы, и называется декомпозицией. Подсистема отличается от простой группы элементов, для которой не выполняется условие целостности.

Последовательное разбиение системы в глубину приводит к получению иерархии подсистем, нижним уровнем которых является элемент. С этой концепцией связано понятие структуры системы.

1.2. Системный подход к описанию объекта и процесса информатизации

Информатизация – это направленный процесс системной интеграции компьютерных средств, информационных и коммуникационных технологий с целью выработки релевантной информации в области ее применения и получения новых общесистемных свойств этой области, позволяющих более эффективно организовать процесс информатизации

Процесс информатизации – продуктивная деятельность человека или группы, целенаправленно осуществляемая в рамках объекта информатизации и имеющая целью достижение конкретного результата, представляющего ценность для потребителя.

Объект информатизации – область деятельности человека или группы, характеризующаяся определенной целью и обладающая свойствами системы, элементами которой являются в том числе люди как субъекты деятельности.

Пример:

Объект информатизации: торговое предприятие.

Процесс информатизации: складской учет товаров.

Соответствующая тема: Спроектировать программную систему складского учета товаров.

Объект информатизации описывается лаконично простым повествовательным техническим текстом и содержит такие сведения:

1. Наименование объекта. Если объектом является организация, указывается ее юридическое название. Если объектом является техническое устройство, указывается его название согласно официальной документации на него.

2. Краткая характеристика объекта в 3–5 простых предложениях, раскрывающая его суть. Здесь уместно дать ссылку на источник этой информации.

3. Структурная схема, иллюстрирующая архитектуру объекта. Если объектом является организация, приводится её организационная диаграмма. Если объектом является техническое устройство, приводится его функциональная структура или алгоритм работы. Уместны и другие иллюстрации, если они значимо дополняют текстовую часть описания.

4. Текстовые комментарии компонентов структурной схемы, в том числе задачи (функции) компонентов и особенности их взаимодействия при функционировании объекта.

5. Основные показатели эффективности функционирования объекта (не более трех), на улучшение которых должна быть направлена информатизация.

Пример исчерпывающего описания объекта представлен в [приложении Б](#).

Процесс информатизации, как правило, выбирается среди функционала объекта информатизации. Укрупнено его можно представить в нотации IDEF0 в виде модели «Черный ящик» (рис. 1).

Если объектом является организация, то процесс информатизации представляет собой *бизнес-процесс*. В любом случае процесс информатизации объединяет в себе несколько задач (функций) объекта. Такую структуру можно представить в виде диаграммы декомпозиции.

Разделение бизнес-процесса на подпроцессы (задачи) производится с учетом целевой функции бизнес-процесса, а деление операции – с учетом целевой функции подпроцесса, исходя из целевой функции бизнес-процесса. Такой принцип упрощает изучение сложных бизнес-процессов и является системным подходом. На рис. 2 представлен принцип декомпозиции, использующийся в системном анализе.

В каждом конкретном случае процессу информатизации дается лаконичное и информативное наименование, в полной мере отражающее суть выполняемых действий на объекте. Именованье задач и операций также выполняются в соответствии с этим принципом.

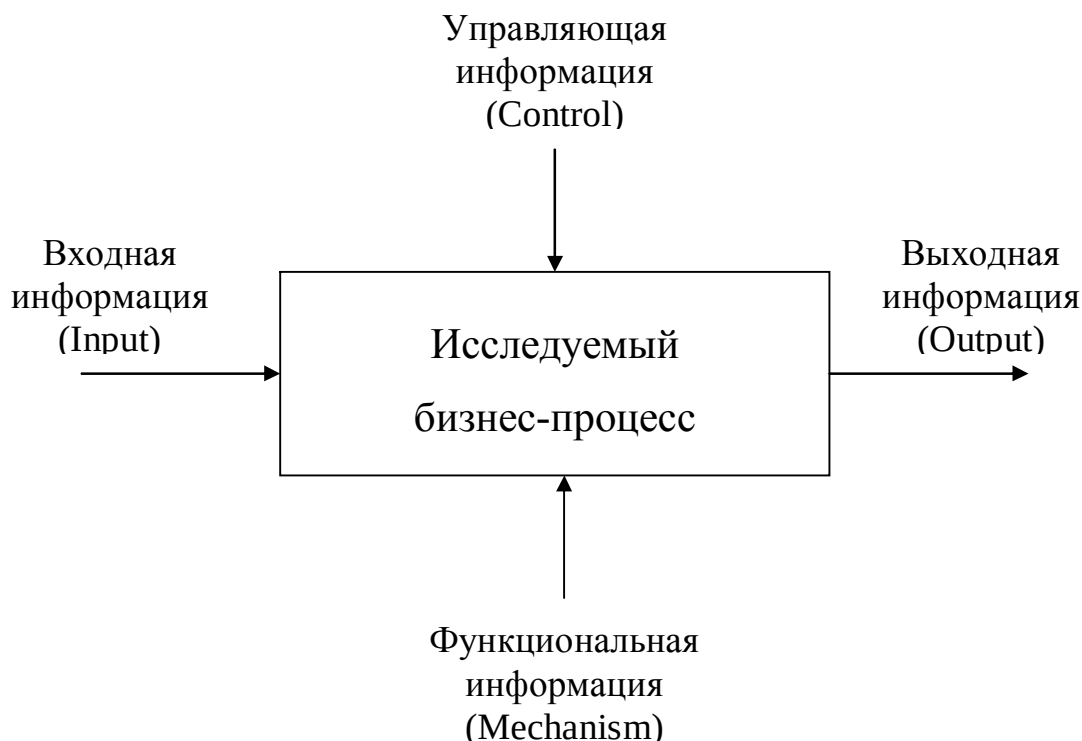


Рис. 1. Обобщенная схема модели «Черный ящик»

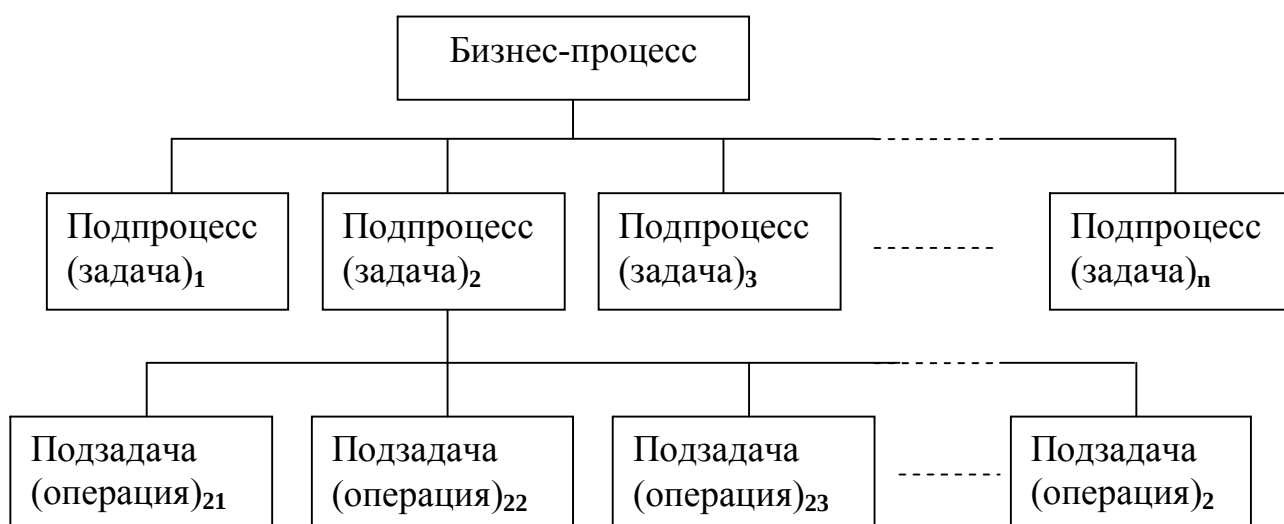


Рис. 2. Принцип декомпозиции бизнес-процесса

При выделении задач следует помнить, что задача программы – это формализованная совокупность действий, выполнение которых приводит к результату заданного вида. Поэтому в качестве задач надо выбирать такие, для которых можно четко сформулировать результат. В данном примере можно выделить такие задачи:

- задача ввода входных данных;
- задача сохранения данных в памяти ЭВМ;
- задача формирования выходных данных;
- задача вычисления некоторого итогового показателя;
- задача статистического анализа данных, и др.

Описание информационных потоков. При выделении задач следует учитывать, что это информационно взаимосвязанный функциональный комплекс, назначение которого – переработка входной информации в выходную. Любой функциональный элемент, будь то процесс, задача или операция, получает на вход некоторую входную информацию (исходные данные, задание) и генерирует некоторую выходную результатную информацию. Возможно также выделение внутренней информации, циркулирующая внутри процесса. Как правило, это промежуточные данные, являющиеся результатами переработки входных инфопотоков и исходными данными для выработки выходных инфопотоков.

Вся информация циркулирует в рамках процесса информатизации в форме информационных потоков (инфопотоков) и требует описания и систематизации. Такое описание уместно представить форме реестров (табл. 1, 2).

Возможные формы представления информации: сигнал, устное сообщение, устное распоряжение, документ, и др. Наименование потока должно адекватно отражать его сущность и соотноситься с его формой представления.

Трудозатраты представляют собой вещественную неотрицательную оценку, выражающую количество часов, которые должен работать 1 человек, чтобы воспринять и обработать входной поток или сформировать и вывести выходной поток. В редких случаях, когда человек не причастен к обработке инфопотока, выставляется оценка 0.

Таблица 1. Реестр входных информационных потоков

№	Наименование и назначение потока	Форма представления	Обработчик (Кто обрабатывает)	Корреспондент (Откуда поступает)	Характеристики обработки	
					Трудозатраты, чел.ч	Периодичность, регламент

Таблица 2. Реестр выходных информационных потоков

№	Наименование и назначение потока	Форма представления	Обработчик (Кто обрабатывает)	Корреспондент (Куда отправляется)	Характеристики обработки	
					Трудозатраты, чел.ч	Периодичность, регламент

Периодичность отражает частоту следования инфопотока. Как правило, даются средние оценки. Наилучший способ определения этой характеристики – получить экспертное мнение специалиста из предметной области. Если такого специалиста нет, то периодичность следует оценить исходя из назначения инфопотока и логики его обработки. К примеру, для оперативной информации периодичность может составлять от 10 раз в сутки (например, заявка на ремонт оборудования) до 100 раз в секунду (например, сигнал от датчика системы реального времени). Для справочной информации периодичность гораздо ниже и может составлять 1 раз в год (обновление штатного расписания) или 10 раз в месяц (сведения о трудоустраиваемом лице). В крайнем случае, если никак не возможно установить приблизительную частоту следования инфопотока, допускается указать «По требованию».

Если формой представления инфопотока является документ, целесообразно представить его макет, либо копию. При этом необходимо кратко описать такие реквизиты документа, смысл которых не является очевидным.

1.3. Модель вариантов использования

Модель прецедентов (модель требований, модель *вариантов использования*) – это базовая диаграмма UML, на которой изображаются отношения между действующими лицами и действиями (прецедентами). Это исходная концептуальная модель системы в процессе ее проектирования и разработки. Создание диаграммы вариантов использования имеет следующие цели:

- определение общих границ и контекста моделируемой предметной области на начальных этапах проектирования ПС;
- формальное представление требований к функциональному поведению (функциональных требований) проектируемой ПС;
- разработка исходной концептуальной модели системы для ее последующей детализации в форме логических и физических моделей в нотации UML;
- подготовка исходной документации для взаимодействия разработчиков системы с ее заказчиками и пользователями.

Система или процесс представляется в форме вариантов использования (прецедентов), с которыми взаимодействуют внешние сущности или актеры (в английской транскрипции – экторы). При этом актером называется любой объект, субъект, система или устройство, взаимодействующие с моделируемой системой извне. Концептуальный характер этой диаграммы проявляется в том, что подробная детализация диаграммы на начальном этапе проектирования имеет отрицательный характер, поскольку предопределяет способы реализации поведения системы – эти аспекты должны быть сознательно скрыты от разработчика на модели прецедентов.

Вариант использования (use case) – внешняя спецификация последовательности действий, которые система или другая сущность могут выполнять в процессе взаимодействия с актерами. Служит для описания сервисов, которые система предоставляет актеру в соответствии с его потребностями без рассмотрения ее внутренней структуры.

Отдельный вариант использования обозначается на диаграмме эллипсом, внутри которого содержится его краткое имя в форме глагольного существительного (рис. 3, а) или глагола (рис. 3, б) с пояснительными словами. Сам текст имени прецедента должен начинаться с заглавной буквы.

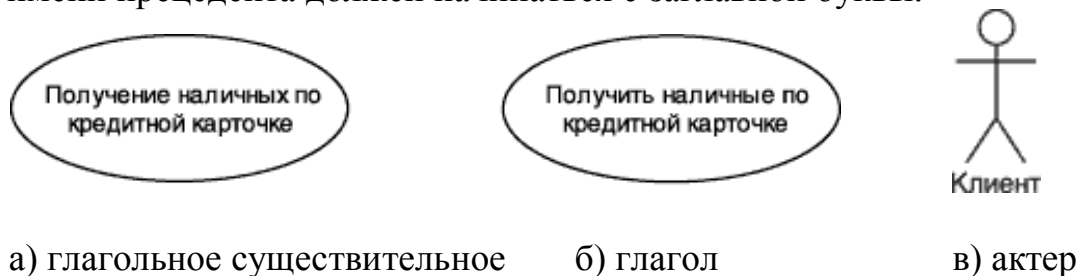


Рис. 3. Графическое обозначение варианта использования и актера

Диаграмма вариантов использования содержит конечное множество вариантов использования, которые в целом должны определять все возможные аспекты ожидаемого поведения системы. Данной модели на всех этапах работы над проектом позволяет не только достичь требуемого уровня унификации обозначений для представления функциональности программы, но и является мощным средством последовательного уточнения требований на основе их итеративного обсуждения со всеми заинтересованными специалистами.

Примерами вариантов использования могут быть следующие действия:

- занесение системы данных о клиенте;
- проверка состояния текущего счета клиента;
- оформление заказа на покупку товара;
- получение информации о кредитоспособности клиента;
- отображение графической формы на экране монитора;
- авторизация пользователя;
- и другие действия, процессы, задачи.

Актер (actor) рассматривается как определенная роль относительно конкретного варианта использования. Стандартным графическим обозначением актера на диаграммах является фигурка человечка, под которой записывается имя актера (рис. 3, в). Имя актера должно быть достаточно информативным с точки зрения семантики. Это может быть:

- наименование должности (продавец, кассир, менеджер, и т.д.)
- технические устройства (датчики, терминалы обслуживания и т.д.);
- другие системы.

Не рекомендуется давать актерам имена собственные или названия моделей конкретных устройств, даже если это с очевидностью следует из контекста проекта, так как одно и то же лицо (устройство) может выступать в нескольких ролях и, соответственно, обращаться к различным сервисам проектируемой системы.

Рассматривая данную диаграмму в качестве модели бизнес-системы, можно ассоциировать ее с "черным ящиком" (рис. 1). При этом справедливы следующие

Правила соответствия моделей вариантов использования и «Черный ящик»:

1. Для обработки каждого входящего инфопотока на модели «Черный ящик» должен быть хотя бы 1 прецедент на модели вариантов использования.
2. Для формирования каждого исходящего инфопотока на модели «Черный ящик» должен быть хотя бы 1 прецедент на модели вариантов использования.
3. Каждому прецеденту на модели вариантов использования должен быть поставлен в соответствие хотя бы 1 инфопоток на модели «Черный ящик».
4. Число и названия актеров на модели вариантов использования должны совпадать с составом инфопотоков «Mechanism» на модели «Черный ящик».

! В случае нарушения хотя бы одного из указанных правил следует внести изменения в модели так, чтобы все 4 правила выполнялись.

Задание

1. Дать характеристику объекта информатизации: наименование, назначение, структура, задачи, действующие лица.
2. Выполнить системное описание заданного бизнес-процесса и выполнить его декомпозицию на подпроцессы (задачи).
3. Построить модель «Черный ящик» и диаграмму вариантов использования UML. Описать построенные модели. Сформировать реестры инфопотоков.
4. Дать характеристику схеме решения задач в ручном режиме и выделить ее недостатки.
5. Обосновать необходимость усовершенствования и развития существующей схемы решения задач за счет создания специального программного обеспечения.

Требования к содержанию отчета

В подразделе 1 дается характеристика заданного объекта информатизации. Приводится его текстовое и графическое описание согласно технологии, рассмотренной в разделе 1.2.

В подразделе 2 выделяется процесс информатизации и ему дается наименование. При этом выполняются следующие обязательные элементы:

- приводится подробное повествовательное описание процесса;
- определяется состав действующих лиц, задействованных в процессе;
- определяется входная и выходная информация, строится структурная схема типа "черный ящик" (рис. 1);
- производится декомпозиция бизнес-процесса на подпроцессы (задачи), строится диаграмма декомпозиции (рис. 2);
- дается общая информация о задачах;
- строится базовая модель вариантов использования (без детализации);
- выполняется описание входных и выходных информационных потоков в виде табл. 1 и 2;
- приводятся правила обработки информации и возможные ограничения;
- определяется нормативно-справочная документация, регламентирующая бизнес-процесс.

На примере объекта информатизации ПАО Банк «ФК Открытие», описание которого представлено в [приложении Б](#), рассмотрим одну из его задач «Проведение операций с иностранной валютой». Тогда процесс информатизации может быть назван так же как и задача, или проще: «Обмен валюты». На рис. 4 и 5 представлены модели «Черный ящик» и вариантов использования.

В целом описание бизнес-процесса должно давать представление о вычислительных задачах, предполагаемых заданным бизнес-процессом, величине информационного массива, подлежащего обработке, численности персонала, занятого в процессе создания специального ПО для решения выделенных задач.

Для более строгого структурирования данного материала рекомендуется использовать табличную форму подачи информации.

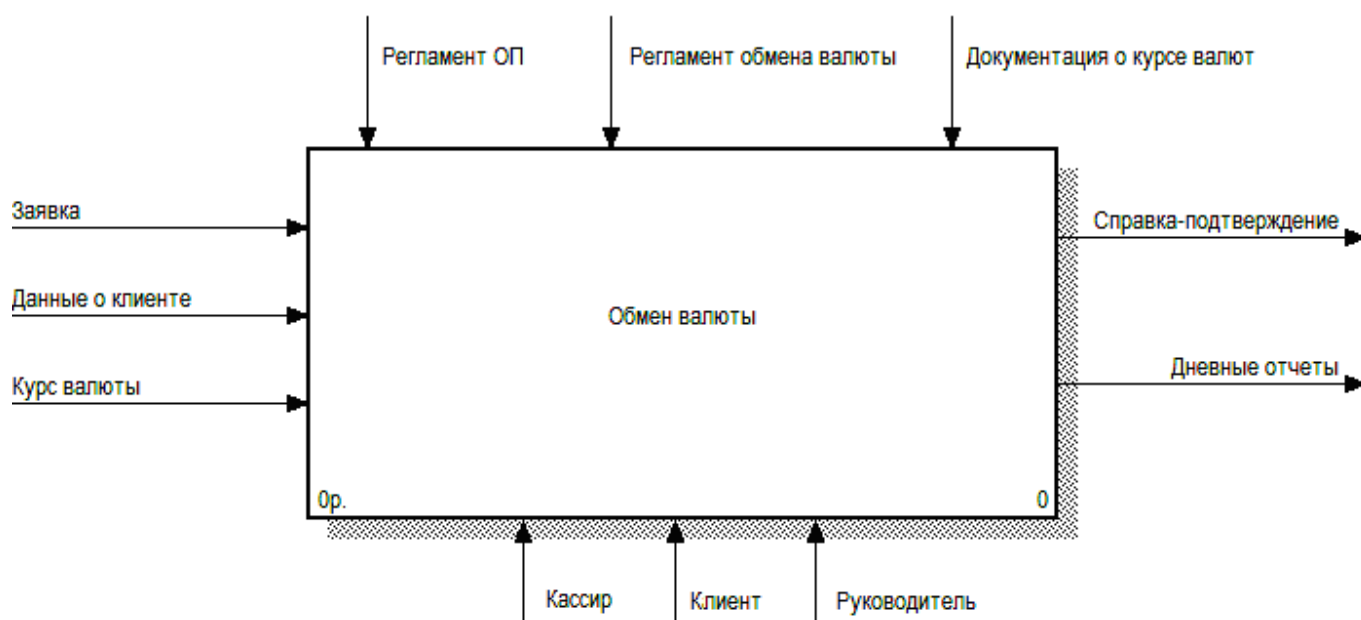


Рис. 4. Пример модели «Черный ящик» для бизнес-процесса обмена валюты

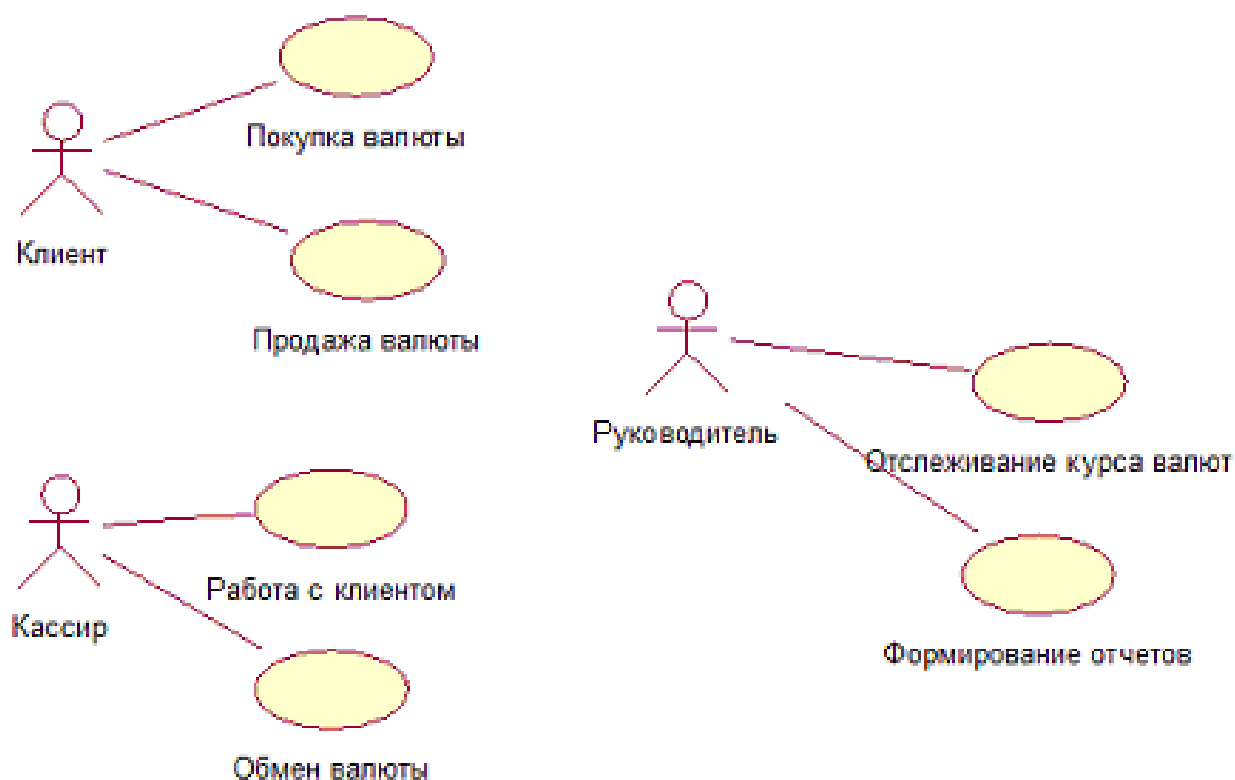


Рис. 5. Пример модели вариантов использования UML для бизнес-процесса обмена валюты

В подразделе 3 в повествовательной форме простыми предложениями описываются основные операции, которые выполняются при сборе и обработке информации, а также связанные с этим действия персонала предприятия без использования специального программного обеспечения. Указываются те

недостатки этой системы, которые приводят к снижению эффективности решения данных задач и бизнес-процесса в целом.

По результатам анализа недостатков дается обоснование необходимости создания специального ПО для автоматизации бизнес-процесса. Излагаются причины, вследствие которых создание программ для решения задач бизнес-процесса является необходимым.

В целом из материала отчета должно быть видно, что представляет собой заданный бизнес-процесс, какие задачи при этом решаются, какие из них выполняются не достаточно эффективно, почему и на каком уровне необходима программная реализация указанных задач.

Контрольные вопросы и упражнения

1. Что такое декомпозиция бизнес-процесса?
2. Какова типовая структура декомпозиции бизнес-процесса?
3. Что такое схема типа "черный ящик"?
4. Что такое документооборот бизнес-процесса?
5. Что такое нормативно-справочная документация, регламентирующая бизнес-процесс?
6. В чем заключаются цели и назначение выбранного бизнес-процесса и какова его динамика?
7. Что такое программное обеспечение (ПО)?
8. Что такое задача?
9. Чем определяется эффективность решения задач ПО?
10. В чем различия между задачей бизнес-процесса и задачей программы?
11. Каковы могут быть основания для создания специального ПО для автоматизации бизнес-процесса?
12. Какие задачи выбранного бизнес-процесса решаются не достаточно эффективно и почему?
13. Какие критерии эффективности могут быть использованы для оценивания эффективности реализации бизнес-процесса?
14. Перечислите задачи в структуре заданного бизнес-процесса. Дайте краткую характеристику одной из задач, включая задействованные документы.
15. Насколько целесообразным является решение об автоматизации выделенных задач бизнес-процесса?
16. Приведите пример правил обработки информации при описании бизнес-процесса. Выполните описание основных операций, которые выполняются при сборе и обработке информации для конкретной задачи бизнес-процесса.
17. Укажите несколько недостатков, которые приводят к снижению эффективности решения задач бизнес-процесса.

Лабораторная работа №2

Тема: Анализ существующих компьютерных разработок.

Цель: ознакомление с существующими разработками подобных программных решений по выбранной теме, приобретение навыков анализа существующих компьютерных разработок.

Задание

1. Выполнить системное описание существующих подобных программных систем (не менее двух), которые могут быть применены к данному объекту управления; выделить основные преимущества и недостатки представленных систем.
2. Выполнить сравнительную характеристику описанных систем. Результаты сравнительного анализа представить в табличной форме. Набор основных показателей для сравнения (подробнее см. [приложение В](#)):
 - назначение системы;
 - эффективность системы;
 - гибкость системы;
 - защищенность системы;
 - живучесть системы;
 - надежность системы;
 - открытость системы;
 - оптимальность использования ресурсов;
 - удобство пользовательского интерфейса системы;
 - стоимость системы (в том числе затраты на тех. поддержку);
 - эргономичность.
3. Сделать вывод о возможности или невозможности использования этих систем на выбранном объекте информатизации.

Требования к содержанию отчета

В подразделе 1 одно за другим приводятся описания ранее разработанных компьютеризированных систем (подсистем), автоматизирующих объекты и процессы, подобные имеющему место в выбранной предметной области. Описание уместно сопровождать следующими иллюстрациями:

- функционально-структурная схема системы;
- обобщенная блок-схема алгоритма функционирования системы;
- экранные формы основных частей пользовательского интерфейса;
- таблицы и графики, отражающие статистические показатели функционирования системы.

Для каждой системы указываются преимущества и недостатки: вообще, а не применительно к выбранному объекту

В подразделе 2 выполняется сравнение представленных систем по нескольким показателям. Сравнение подкреплять количественными показателями (например, сроки внедрения, объем дискового пространства, кол-во единиц техники, стоимость

программного обеспечения и т.п.) и статистическими оценками в виде таблиц, диаграмм, графиков.

В подразделе 3 делается заключение о возможности применения рассмотренных систем к выбранному объекту и процессу информатизации. Оценивается степень этой возможности на предмет удобства и скорости настройки системы на данную предметную область, а также оптимальности их внедрения с учетом затрат на дальнейшее обслуживание и тех. поддержку.

В целом из материала отчета должно быть видно, какие из систем, автоматизирующих подобные выбранным объекты и процессы, существуют на отечественном и зарубежном рынке, и вообще в мире. Какой комплекс задач они позволяют решить, насколько оперативны и эффективны получаемые решения и насколько они соответствуют целям, провозглашенным в лаб. работе №1.

Контрольные вопросы

(ответы должны подкрепляться количественными оценками)

1. Насколько широк спектр существующих автоматизированных систем, которые уместно применить к выбранному объекту информатизации?
2. Какая из рассмотренных систем наиболее близка для применения на выбранном объекте информатизации, и почему?
3. Насколько глубокие изменения необходимо произвести в той или иной из описанных систем для ее использования в другой предметной области?
4. Как быстро произойдет окупаемость той или иной из описанных систем при их внедрении на выбранном объекте?
5. Почему какую-либо из рассмотренных систем целесообразно применить на выбранном объекте, а почему нецелесообразно?

Лабораторная работа №3

Тема: Техническое задание на создание программного продукта.

Цель: Освоение методики предварительного анализа разрабатываемой программы; освоение задач формулирования функциональных и нефункциональных требований к программной реализации отдельных задач и к программе в целом; выработка навыков разработки технического задания.

Задание

1. Установить назначение и общую цель создания программы.
2. Определить структуру программы и состав функциональных задач.
3. Разработать функциональные требования к программе:
 - требования к входным и выходным данным;
 - требования к программной реализации задач;
 - специальные требования к математическому обеспечению программной реализации задач;
4. Разработать модель требований в нотации UML
5. Разработать требования к информационному обеспечению (к базе данных).
6. Разработать требования к инструментальному программному обеспечению (к системе управления базой данных (СУБД), к средству разработки программ (IDE), средствам автоматизированного проектирования ПО)
7. Установить нефункциональные требования к программе. Дать не менее 5 наиболее очевидных для данной системы требований из [приложения В](#).

Требования к содержанию отчета

В подразделе 1 описывается назначение подсистемы и цели ее создания. При описании назначения ПС указывают вид деятельности, которая автоматизируется (учет, расчет, управление, диагностика, проектирование и т.п.) и перечень объектов автоматизации, на которых предполагается ее использовать. При описании цели создания подсистемы приводят наименование и необходимые значения технических, технологических, производственно-экономических или других показателей объекта автоматизации, которые должны быть достигнуты в результате создания ПС, и указывают критерии оценки достижения указанных целей.

В подразделе 2 указывается перечень задач, программную реализацию которых предполагается осуществить. Разрабатывается статическая объектная модель будущей программы в виде диаграммы вариантов использования и диаграммы классов. Диаграммы реализовать в пакете программ Rational Rose, Visio или Altova UModel.

В подразделе 3 при формулировании функциональных требований к программе необходимо учитывать, что потребуется обеспечить их выполнение при программировании соответствующих задач.

Изложение функциональных требований разбивается на несколько пунктов в зависимости от количества задач, например:

3.1 Требования к задаче “...”

3.2 Требования к задаче “...”

3.3 Требования к задаче “...”

В каждом пункте указывается полное название задачи. Кратко излагается, какие в точности действия программа должна выполнить для реализации данной задачи, в форме словесного или формульно-словесного описания алгоритма. Кроме того, по каждой задаче могут быть указаны следующие требования:

- к временному регламенту реализации каждой задачи;
- к качеству реализации каждой задачи;
- к выходным данным;
- к входным данным;
- к преобразованию входной информации к машиночитаемому виду;
- к возможной одновременности выполнения нескольких задач;
- к достоверности результатов.

При описании требований к входным данным приводится: перечень и описание входных данных (идентификатор, форма представления, сроки и частота поступления), перечень и описание структурных единиц информации входных сообщений или ссылка источники этих данных. В описании для каждой структурной единицы информации входных сообщений следует указывать:

- наименование;
- необходимую точность ее числового значения (при необходимости);
- источник информации (документ, видеокادر, устройство, кодограмма, информационная база на машинных носителях и т.д.);
- идентификатор источника информации.

Требования к программной реализации задач должны содержать:

- требования к организации хранения данных в виде базы данных;
- предварительное описание (структуру) интерфейса для доступа к реализации отдельных задач и подзадач;
- требования к методу программирования, к структуре кода, к именованию программных и информационных объектов.

При формулировке специальных требований к математическому обеспечению программной реализации задач приводят требования к составу, области применения (ограничение) и средств использования в системе математических методов и моделей, типичных алгоритмов и алгоритмов, которые подлежат разработке.

В подразделе 4 выполняется построение модели требований в нотации UML. Предусмотреть расширение и включение вариантов использования. Пример модели требований в нотации UML показан на рис. 6.

В подразделе 5 указываются требования к базе данных ПС, которые могут включать следующие пункты:

- требования к организации данных, которые должны сохраняться в ПС;
- отсутствие дублирования информации и сокращение чрезмерности данных, поддержка целостности, достоверности и актуальности данных;
- низкая стоимость хранения и использования данных;

В подразделе 6 указываются:

- требования к операционной среде;
- требования к инструментальным средствам программной инженерии, обеспечивающих разработку ПО (CASE-средства и средства объектно-ориентированного моделирования);
- требования к инструментальным средствам разработки ПО;
- требования к использованию готовых программных пакетов;
- требования к вспомогательным программным средствам (сервисные программы и утилиты).

В подразделе 7 указывают нефункциональные требования к разрабатываемой программе. Рекомендуется выбрать не менее 5 показателей, наиболее соответствующих специфике разрабатываемой ПС. Формулировки нефункциональных требований должны быть семантически и терминологически связаны с функциональными требованиями. Требования к их формулировкам такие же, как и к формулировкам функциональных требований (см. подраздел 3.3).

Все требования к подсистеме согласовываются с руководителем. ГОСТ на содержание технического задания на создание программ приведен в [приложении Г](#).

Контрольные вопросы и упражнения

1. Что такое техническое задание и какова его структура?
2. Для чего и зачем разрабатываются компьютерные программы?
3. Какие положительные результаты могут быть получены в процессе использования компьютерной программы?
4. Что подразумевает программная реализация задач бизнес-процесса?
5. Каково назначение разрабатываемой компьютерной программы?
6. Назовите цели, в соответствии с которыми разрабатывается программа.
7. Дайте определение функционально-структурной и объектной модели компьютерной программы. Укажите принципиальные различия между этими моделями.
8. Перечислите базовые объектные модели компьютерной программы и компьютерные средства их реализации.
9. Что такое функциональные и нефункциональные требования к компьютерной программе?
10. Принципы формулировки требования к компьютерной программе при разработке технического задания.
11. Укажите основные аспекты формулирования функциональных требований к программной реализации задач бизнес-процессов.
12. Какие виды обеспечения компьютерной программы разрабатываются при ее создании?
13. Что такое прикладное обеспечение компьютерной программы?
14. Какие требования предъявляются к входным и выходным данным при программной реализации задач бизнес-процессов?

15. Какие требования предъявляются собственно к программной реализации задач бизнес-процессов?
16. Какие требования предъявляются к прикладному математическому обеспечению при программной реализации задач бизнес-процессов?
17. Улучшение каких технических, технологических, производственно-экономических или других показателей бизнес-процесса может быть достигнуто в результате создания и использования компьютерной программы?
18. Какова численность оперативного персонала, задействованного в автоматизируемых задачах до и после предполагаемого внедрения компьютерной программы?
19. Сформулировать функциональные и нефункциональные требования к программному приложению, реализующему поиск данных в односвязном списке тремя разными способами. Разработать объектные модели, описывающие работу программы.

Лабораторная работа №4

Тема: Проектирование функциональной структуры программного продукта: функционально-ориентированный подход.

Цель: изучение методики функционально-ориентированного подхода программной инженерии для разработки и описания функциональности разрабатываемого программного обеспечения.

Теоретические положения

4.1 Функционально-ориентированный подход к проектированию ПС

Функционально-ориентированный подход рассматривает объект исследования, как набор функций, преобразующий поступающий поток информации в выходной поток. Процесс преобразования информации потребляет определенные ресурсы. Принципиальное отличие данного подхода от других подходов заключается в четком отделении функций (методов обработки данных) от самих данных.

Распространенной методикой реализации функционально-ориентированного подхода является IDEF0, являющаяся следующим этапом развития графического языка описания функциональных систем SADT (Structured Analysis and Design Technique). Целью методики является построение функциональной схемы исследуемой системы, описывающей все необходимые процессы с точностью, достаточной для однозначного моделирования деятельности системы.

В основе методологии лежат четыре основных понятия: функциональный блок, интерфейсная дуга, декомпозиция, глоссарий.

Функциональный блок (Activity Box) представляет собой некоторую конкретную функцию в рамках рассматриваемой системы. По требованиям стандарта название каждого функционального блока должно быть сформулировано в глагольном наклонении (например, "производить услуги"). На диаграмме функциональный блок изображается прямоугольником с текстом названия внутри (см. рис. 1, 4).

Интерфейсная дуга (Arrow) отображает элемент системы, который либо обрабатывается функциональным блоком, либо является результатом этой обработки, либо оказывает иное влияние на *функцию*, представленную данным функциональным блоком. С их помощью отображают объекты, в той или иной степени определяющие процессы, происходящие в системе. Это могут быть элементы реального мира (детали, вагоны, сотрудники и т.д.) или инфопотоки (документы, данные, инструкции и т.д.). В зависимости от того, к какой из сторон функционального блока подходит дуга, она носит название "входящей", "исходящей" или "управляющей".

Любой функциональный блок по требованиям стандарта должен иметь, по крайней мере, одну управляющую интерфейсную дугу и одну исходящую.

Декомпозиция (Decomposition) является основным понятием стандарта IDEF0. Принцип декомпозиции применяется при разбиении сложного процесса на

составляющие его функции. При этом уровень детализации процесса определяется непосредственно разработчиком.

Декомпозиция позволяет постепенно и структурированно представлять модель системы в виде иерархической структуры отдельных диаграмм, что делает ее менее перегруженной и легко усваиваемой.

Модель IDEF0 всегда начинается с представления системы как единого целого – одного функционального блока с интерфейсными дугами, простирающимися за пределы рассматриваемой области. Такая диаграмма с одним функциональным блоком называется контекстной диаграммой.

В процессе декомпозиции базовый функциональный блок подвергается детализации на другой диаграмме. Получившаяся диаграмма второго уровня содержит функциональные блоки, отображающие главные подфункции функционального блока контекстной диаграммы, и называется дочерней (Child Diagram) по отношению к нему, а каждый функциональный блок дочерней диаграммы – дочерним блоком (Child Box). В свою очередь, функциональный блок-предок называется родительским блоком по отношению к дочерней диаграмме (Parent Box), а диаграмма, к которой он принадлежит – родительской диаграммой (Parent Diagram). Каждая из подфункций дочерней диаграммы может быть далее детализирована путем аналогичной декомпозиции соответствующего ей функционального блока. В каждом случае декомпозиции функционального блока все интерфейсные дуги, входящие в данный блок или исходящие из него, фиксируются на дочерней диаграмме. Этим достигается структурная целостность IDEF0-модели.

Обычно IDEF0-модели несут в себе сложную и концентрированную информацию. Для того чтобы ограничить их перегруженность и сделать удобочитаемыми, в стандарте приняты соответствующие ограничения сложности:

- количество функциональных блоков на диаграмме – от 3 до 6;
- количество интерфейсных дуг на один функциональный блок не более 5;
- любой функциональный блок должен иметь не менее одной управляющей (Control, Mechanism) и одной исходящей (Output) дуги;
- компоновка блоков и интерфейсных дуг с подписями должна обеспечивать максимальную читабельность диаграммы.

Преимущества ФОП выражаются в наглядности графического языка, структурной целостности модели и преемственности при организации новых проектов. Недостатком его является сложность проектирования, возрастающая вместе со сложностью проектируемой системы.

Основной недостаток функционального моделирования связан с отсутствием явных средств для объектно-ориентированного представления моделей сложных систем. Некоторые аналитики отмечают важность знания и применения нотации IDEF0, однако отсутствие возможности реализации соответствующих графических моделей в объектно-ориентированном программном коде существенно сужают диапазон задач, решаемых с ее помощью.

4.2. Пример функционального моделирования ПС

Начинать функциональное моделирование ПС следует с перечисления её автоматизированных функций. Перечисление автоматизированных функций выполняется по пунктам простыми повествовательными предложениями. Объем текста описания для несложной информационной функции ПС обычно составляет 30–50 слов. В случае описания сложной вычислительной или управляющей функции ПС слов может быть больше в разы. При этом в описание при необходимости могут быть включены формулы, таблицы и дополнительные иллюстрации. Так, например, в описание функции операции обмена валюты может быть представлена формула для расчета суммы к выдаче.

При построении контекстной диаграммы ПС за основу принимается разработанная при выполнении [лабораторной работы №1](#) структурная схема типа «черный ящик» с обозначенными входными (Input) выходными (Output) данными, а также списков нормативно-справочной документации (Control) и экторов, задействованных в бизнес-процессе (Mechanism) (подробнее см. рис. 1). Пример контекстной диаграммы, выполненной при помощи CASE-средства BPWin, представлен на рис. 4 в требованиях к содержанию отчета о выполнении лабораторной работы №1.

Следует иметь в виду, что при выполнении лабораторной работы №1 моделью черного ящика был описан процесс информатизации как есть. На этапе постановки задачи происходит переосмысление процесса информатизации и некоторые его задачи проектируемой ПС реализуются полностью автоматически. В таком случае внешнее дополнение ПС на модели черного ящика может отличаться от исходной модели процесса информатизации. Например, взяв за основу модель, показанную на рис. 4, можно заметить, что эктор «Клиент» является лишним, так как не является пользователем ПС. К тому же очевидно, что эти данные о курсах валюты не вводятся в ПС как входные ни одним из действующих лиц. Вместо этого данные о курсах валют по логике работы программы извлекаются из корпоративной базы данных непосредственно при расчете суммы к выплате. Поэтому контекстная диаграмма для ПС обмена валют будет выглядеть так, как показано на рис. 7.

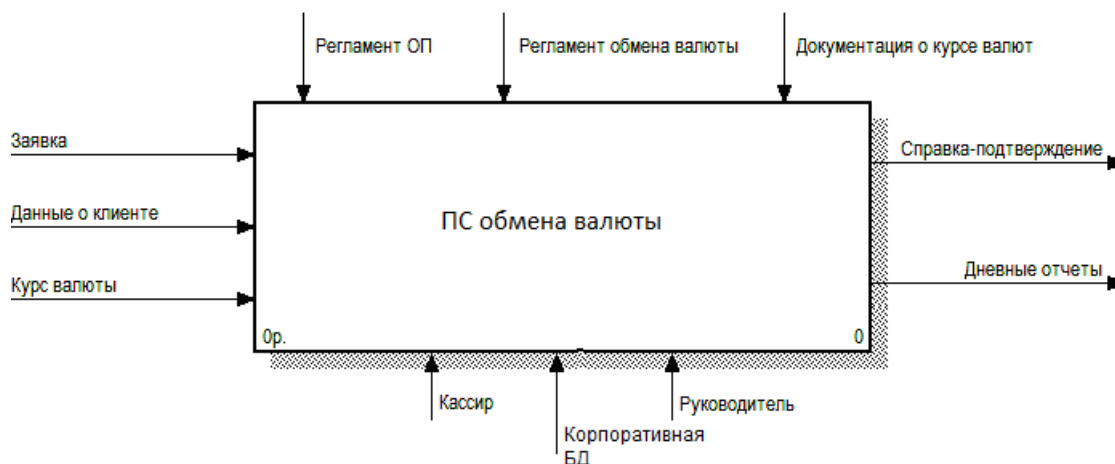


Рис. 7. Контекстная диаграмма ПС выполнения операции обмена валюты, выполненная в BPWin

Диаграмма декомпозиции такой контекстной диаграммы показана на рис. 8. Стрелка эктора «Корпоративная БД» отражает механизм автоматического запроса курса валюты при выполнении функции «Операция обмена валюты» (см. блок 2 на рис. 8).

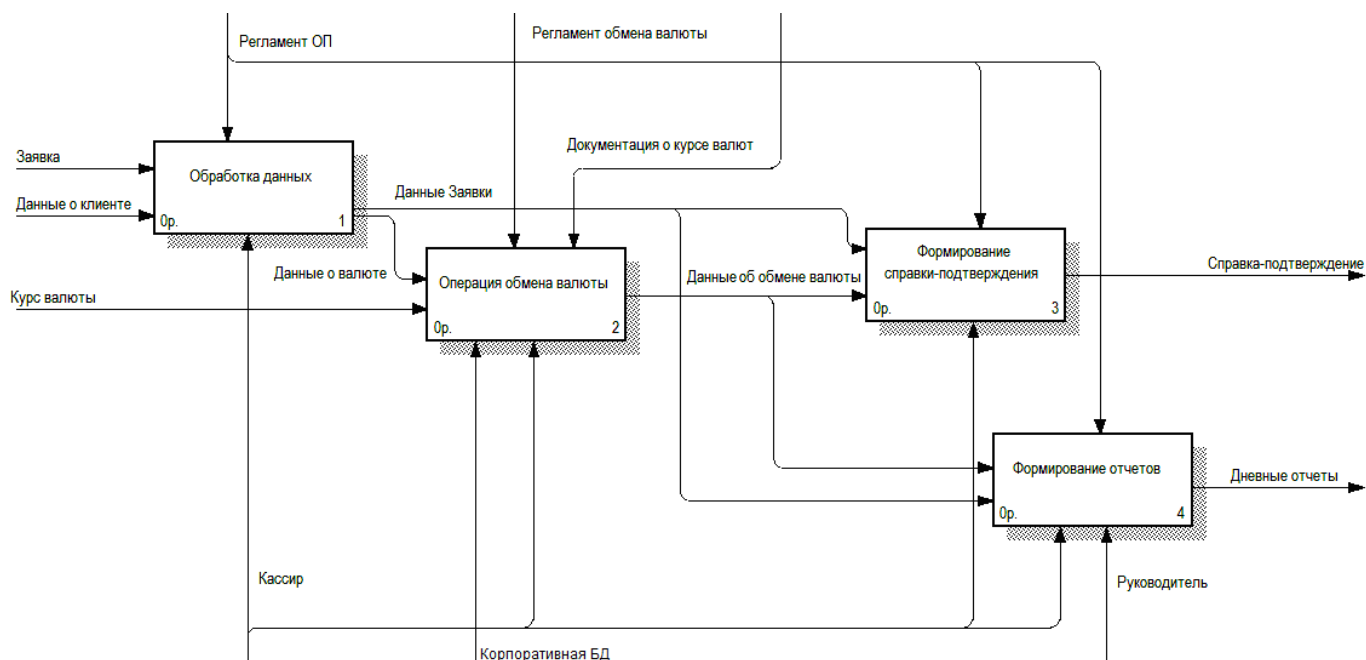


Рис. 8. Диаграмма декомпозиции ПС обмена валюты, выполненная в BPWin

Дальнейшая декомпозиция до уровня операций производится в отдельности для каждого из четырех подпроцессов – блоков диаграммы, показанной на рис. 8. При это следует руководствоваться следующими правилами:

- любая дуга может иметь ответвление, как показано на рис. 8;
- слияние дуг не допускается;
- все дуги должны быть именованы;
- во всей иерархии диаграмм не должно быть дуг с одинаковыми именами.

Задание

1. Построить функциональную модель разрабатываемого ПО в виде контекстной диаграммы в нотации IDEF0 при помощи пакета BPWin.
2. На основе контекстной диаграммы построить диаграмму декомпозиции A0 на дочерние подпроцессы (задачи).
3. Для всех функциональных блоков диаграммы A0 построить диаграммы декомпозиции A2 на подзадачи. По согласованию с преподавателем некоторые блоки могут не декомпонироваться в виду тривиальности их функционала.

Требования к содержанию отчета

В подразделе 1 выполняется перечисление автоматизированных функций ПС, на основе которого строится контекстная диаграмма в нотации IDEF0. Построение выполняется при помощи CASE-средства, например, пакета программ BPWin 4.1 или Ramus Educational (<http://softrare.ru/windows/ramus>).

Для перечисления автоматизированных функций ПС за основу может быть взят материал анализа предметной области из отчета о выполнении [лабораторной работы №1](#). При этом описание автоматизированных функций ПС должно соответствовать функциональным требованиям к ПС, сформулированным в техническом задании при выполнении [лабораторной работы №3](#). Если перечисление функций ПС не соответствует функциональным требованиям, следует модифицировать либо описание автоматизированных функций, либо формулировки функциональных требований.

В подразделе 2 также при помощи CASE-средства BPwin 4.1 строится диаграмма декомпозиции A0. Дочерние процессы диаграммы должны в точности соответствовать задачам, заявленным при формулировании функциональных требований к ПО ([лабораторная работа №3](#)). Связи (интерфейсные дуги) диаграммы должны учитывать функциональные требования к задачам, в частности к входной и выходной информации, к преобразованию данных, и др. При этом следует придерживаться требований ГОСТ.

В подразделе 3 выполняется дальнейшая декомпозиция задач бизнес-процесса на подзадачи – операции. При этом следует опираться на материал подраздела 2 лабораторной работы №1. В качестве операций следует определять такие, для которых можно четко сформулировать результат (Output). Отдельно рассматриваются действия, которые выполняются программными или техническими средствами, и человеком (Mechanism).

Каждая диаграмма A2 должна отражать все действия, направленные на программную обработку и хранение входной информации, а также удовлетворять вышеуказанным требованиям ГОСТ.

Контрольные вопросы и упражнения

1. В чем заключается функционально-ориентированный подход к разработке и описанию функциональности компьютерной программы?
2. Каково назначение и структура функциональной модели компьютерной программы?
3. Что такое перечисление автоматизированных функций компьютерной программы?
4. Чем отличаются модели черного ящика, построенные при выполнении лабораторных работ 1 и 4?
5. Что такое нотация IDEF0?
6. Какова структура функциональной модели в нотации IDEF0?

7. В чем заключается принцип «черного ящика» и принцип декомпозиции при построении функциональной модели в нотации IDEF0?
8. Перечислите основные правила построения диаграмм в нотации IDEF0.
9. Как выражаются функциональные требования к компьютерной программе на диаграммах в нотации IDEF0?
10. Какие существуют ограничения на декомпозицию в нотации IDEF0?
11. Как документируются диаграммы в нотации IDEF0?
12. Выполните функциональное моделирование работы компьютерной программы в нотации IDEF0.

Лабораторная работа №5

Тема: Проектирование функциональной структуры программного продукта: объектно-ориентированный подход.

Цель: изучение методики объектно-ориентированного подхода программной инженерии для разработки и описания функциональности разрабатываемого программного обеспечения.

Методические указания к построению диаграмм UML прилагаются в виде отдельного документа. Использовать CASE-средство ROSE ENTERPRISE либо [STAR UML](#).

Задание

1. Проанализировать описание функционирования программной системы, разработанного при выполнении [лабораторной работы №4](#), на предмет выявления набора абстракций предметной области проектируемой ПС. В качестве предварительных кандидатов в абстракции принять подлежащие, выделенные из текста анализируемого потока событий.

2. Разделить выделенные абстракции на три типа: абстракции сущности, абстракции поведения, абстракции интерфейсы. Для каждой абстракции указать ее класс согласно следующей классификации (табл. 4). Результат представить в виде табл. 5.

Таблица 4. Классификация абстракций

Класс	Описание
Роли	Роли, которые исполняют пользователи, работающие с системой
Предметы	Осязаемый материальный объект или группа объектов
Места, локации	Области, связанные с людьми или предметами, существенные для работы системы
Инструменты	Средства деятельности, используемые определенными ролями
Организации	Формально организованная совокупность людей, ресурсов, оборудования, инструментов, которая имеет определенную цель и существование которой в целом не зависит от людей
Концепции	Принципы и идеи, сами по себе неосязаемые, но предназначенные для организации деятельности и/или общения, или же для наблюдения за ними
События	Нечто случающееся с чем-то/кем-то в заданное время или последовательно. Происшествия, которые должны быть зафиксированы
Показатели	Характеристики, определяющие деятельность, либо ее отдельные процессуальные компоненты
Устройства	Устройства, с которыми взаимодействует система
Другие системы	Внешние системы, с которыми взаимодействует проектируемая система

Таблица 5. Абстракции подсистемы

№	Абстракция	Тип	Класс	Описание
---	------------	-----	-------	----------

3. Проанализировать поведение выделенных абстракций. Выделить возможное поведение каждой абстракции в пределах функциональности проектируемой ПС, представленной моделью требований UML (рис. 6). Заполнить таблицу 6.

Таблица 6. Абстракции подсистемы и их поведение

№	Абстракция	Требование согласно модели UML	Описание поведения
---	------------	--------------------------------	--------------------

4. Построить диаграмму классов UML (class diagram), указывая при этом лишь имена классов без указания свойств класса. Пример диаграммы приведен на рис. 9.

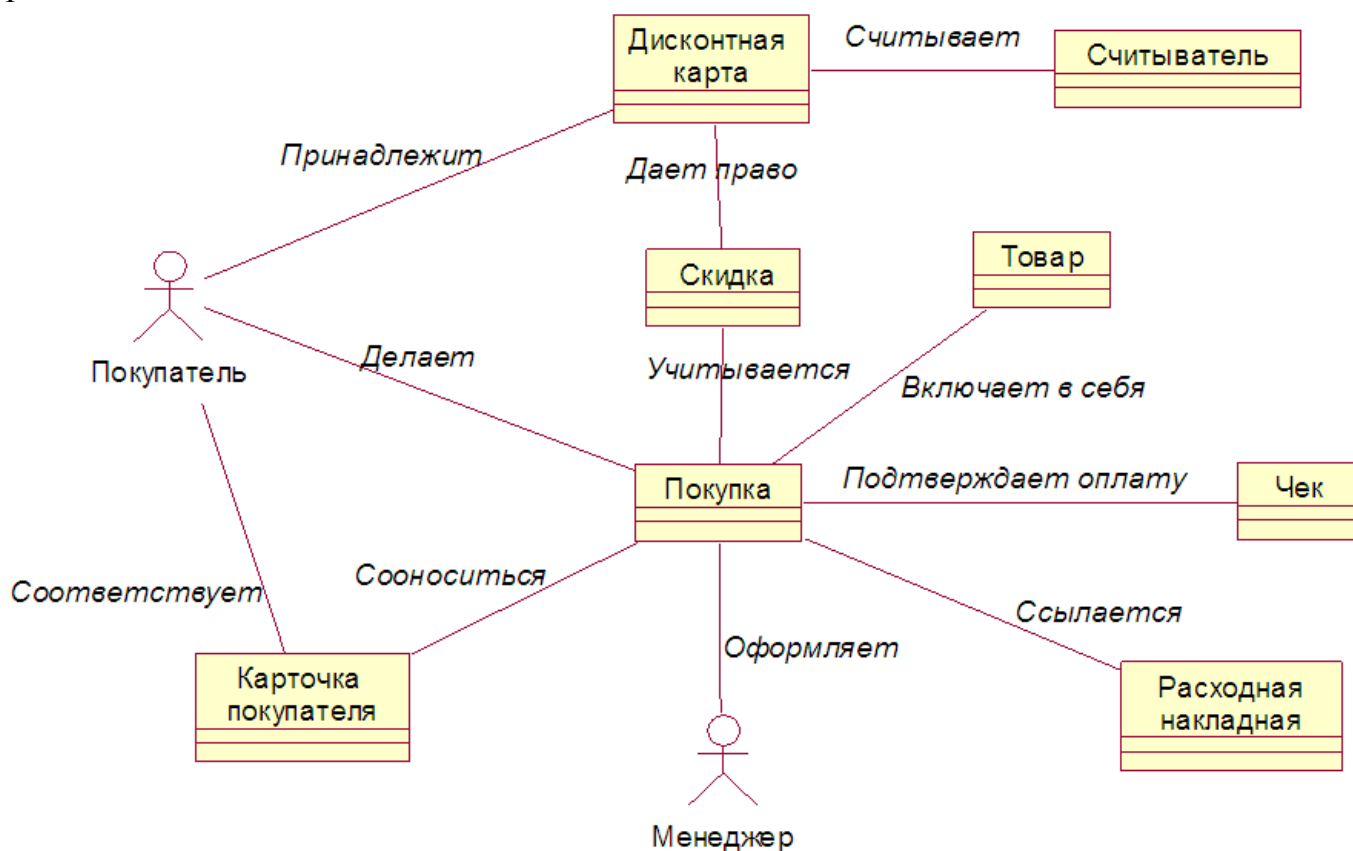


Рис. 9. Пример диаграммы классов UML

5. На основе анализа описания предметной области, разработанного при выполнении [лабораторной работы №1](#), выявить атрибуты и операции классов. Заполнить секции атрибутов и операций классов.

6. Выбрать в модели классов такой класс, которых характеризуется наиболее частой сменой состояний, и построить для него диаграмму состояния (statechart diagram).

7. На основе анализа функциональных моделей, разработанных при выполнении [лабораторной работы №4](#), для каждого из базовых вариантов

использования построить диаграмму деятельности (activity diagram). Для вариантов использования, с которыми связаны несколько действующих лиц, диаграмму деятельности построить в виде дорожек с привязкой к исполнителям отдельных операций варианта использования.

8. Для каждого варианта использования выделить список объектов участвующих во взаимодействии в этом прецеденте, заполнить таблицу 7.

Таблица 7. Список объектов для каждого потока событий

№ п.п.	Прецедент	Объект	Описание объекта
--------	-----------	--------	------------------

9. Построить диаграммы последовательности (sequence diagram) для перечисленных прецедентов. Для простой ПС может быть построена одна диаграмма для всех объектов из табл. 7.

10. Для вариантов использования, характеризующихся сложным поведением, построить кооперативные диаграммы (collaboration diagram).

Требования к содержанию раздела

В подразделе 1 приводится описание функционирования программы в виде сценария или потока событий. При этом следует опираться на статическую функциональную структуру ПО в виде диаграммы требований UML, построенной в при выполнении лабораторной работы №3. Необходимо выделить основные абстракции, их атрибуты и поведение, и описать их в виде табл. 4 и 5. Также представить диаграмму классов с указанием лишь их имен.

В подразделе 2 отражается построение полной модели классов с указанием атрибутов (с указанием типов данных) и операций (с указанием полных сигнатур).

В подразделе 3 определяется класс, которых характеризуется наиболее нетривиальным поведением (наиболее частой сменой состояний). Как правило, это класс-сущность, отражающий одну из центральных сущностей предметной области (для системы управления продажами – это товар, для системы бронирования билетов – билет, для аналитической системы – предмет анализа, и т.п.), с которой выполняются действия по ходу автоматизируемого бизнес-процесса. Для данного класса строится диаграмма состояний UML и выполняется её описание по тексту. Диаграмма состояний должна иметь начальное и конечное состояние (конечных состояний может быть несколько).

В подразделе 4 отражается построение одной или нескольких диаграмм деятельности, которые должны покрывать все функциональные требования и, соответственно, все варианты использования, представленные на модели требований UML, построенной в при выполнении лабораторной работы №3. При этом следует строго придерживаться принципов разветвления и слияния потоков функционирования. Все диаграммы должны быть описаны по тексту подраздела.

В подразделе 5 отражается построение диаграмм взаимодействия UML. При этом необходимо построенными моделями последовательности покрыть все классы и особенности их взаимодействия с другими классами и действующими лицами,

которые представлены на диаграмме требований UML, построенной при выполнении лабораторной работы №3. Для не менее чем одной диаграммы последовательности дополнительно построить диаграмму кооперации UML. Все диаграммы должны быть описаны по тексту подраздела.

Контрольные вопросы

1. В чем заключается метод объектно-ориентированного анализа?
2. В чем заключается метод объектно-ориентированного проектирования?
3. В чем заключается метод перечисления автоматизированных функций?
4. Что такое поток событий? Какова технология его построения?
5. Как выполняется идентификация абстракций?
6. Что такое абстракция поведения?
7. Что такое диаграмма классов UML?
8. Какие секции имеет класс в нотации UML?
9. Какие выделяются типы классов UML? В чем их принципиальное различие?
10. Какие отношения между классами устанавливаются на диаграмме классов UML?
11. В чем заключается установление отношений между классами на диаграмме классов UML?
12. Как соотносятся между собой базовые диаграммы UML: диаграмма требований и диаграмма классов?
13. Как соотносятся между собой базовые диаграммы UML и диаграммы в нотации IDEF0?
14. В чём отличия диаграммы вариантов использования UML от диаграммы требований UML.
15. Какие CASE-средства могут быть использованы для объектно-ориентированного проектирования ПО?
16. Каково назначение диаграмм поведения UML?
17. Перечислите диаграммы поведения UML, опишите их особенности.
18. Как взаимосвязаны диаграммы поведения UML и базовые диаграммы UML?
19. Каков порядок построения диаграмм UML?
20. Откуда берутся исходные данные для построения диаграммы состояния UML?
21. Откуда берутся исходные данные для построения диаграммы деятельности UML?
22. Откуда берутся исходные данные для построения диаграмм взаимодействия UML?
23. Какие диаграммы UML входят в модель программного продукта на языке UML?
24. Сколько и каких диаграмм поведения должно быть построено, если диаграмма прецедентов состоит из трех базовых прецедентов?

Лабораторная работа №6

Тема: Проектирование базы данных программной системы.

Цель: изучение программных средств для разработки моделей информационной базы ПС, проработка методов нормализации отношений в БД, приобретение навыков применения CASE-средства ERwin для моделирования базы данных ПС.

Теоретические положения

6.1. CASE-средство ERwin для моделирования баз данных

Выбор средств проектирования программного обеспечения является одной из самых важных задач при разработке любого программного обеспечения. В настоящее время для проектирования различных систем повсеместно используют CASE-средства.

Термин CASE (Computer Aided Software Engineering) используется в настоящее время в весьма широком смысле. Первоначальное значение термина CASE, ограниченное вопросами автоматизации разработки только лишь ПО, в настоящее время приобрело новый смысл, охватывающий процесс разработки сложных ПС в целом. Теперь под термином CASE-средства понимаются программные средства, поддерживающие процессы создания и сопровождения ПС, включая анализ и формулировку требований, проектирование специального ПО (приложений) и баз данных, генерацию кода, тестирование, документирование, обеспечение качества, конфигурационное управление и управление проектом, а также другие процессы. Таким образом, CASE-средства вместе с системным ПО и техническими средствами образуют полную среду разработки ПС.

Потребности разработчиков и проектировщиков нередко выходят за границы возможностей обычных средств разработки. Поэтому CASE-средства необходимы, например, для осуществления логического моделирования данных, объектного моделирования, а иногда даже для обратного проектирования бизнес-процессов.

Таким образом, разработка ПС с использованием CASE-технологий является наиболее эффективным методом и имеет массу преимуществ, как для пользователей, так и для разработчиков.

Компания CA Technologies является разработчиком программных систем и решений управления ИТ-средой, который обладает опытом работы со всеми ИТ-средами, от больших ЭВМ и распределенных сред до виртуальных и облачных сред. Данная компания занимается разработкой CASE-средства ERwin Data Modeler, которое мы будем использовать при создании базы данных ПС.

ERwin Data Modeler (или просто [ERwin](#)) – CASE-средство для проектирования и документирования баз данных, которое позволяет создавать, документировать и сопровождать базы данных, хранилища и витрины данных. Модели данных помогают визуализировать структуру данных, обеспечивая эффективный процесс организации, управления и администрирования таких

аспектов деятельности предприятия, как уровень сложности данных, технологий баз данных и среды развертывания.

ERwin предназначен для всех компаний, разрабатывающих и использующих базы данных, для администраторов баз данных, системных аналитиков, проектировщиков баз данных, разработчиков, руководителей проектов. ERwin Data Modeler позволяет управлять данными в процессе корпоративных изменений, а также в условиях стремительно изменяющихся технологий.

ERwin позволяет наглядно отображать сложные структуры данных. Удобная в использовании графическая среда ERwin Data Modeler упрощает разработку базы данных и автоматизирует множество трудоемких задач, уменьшая сроки создания высококачественных и высокопроизводительных транзакционных баз данных и хранилищ данных.

ERwin имеет мощные средства визуализации модели, такие, как использование различных шрифтов, цветов и отображение модели на различных уровнях, например, на уровне описания сущности, на уровне первичных ключей сущности и т.д. Эти средства ERwin значительно помогают при презентации модели в кругу разработчиков системы или сторонним лицам.

Обычно разработка модели базы данных состоит из двух этапов: составление логической модели и создание на ее основе физической модели. ERwin полностью поддерживает такой процесс, он имеет два представления модели: логическое (logical) и физическое (physical). Таким образом, разработчик может строить логическую модель базы данных, не задумываясь над деталями физической реализации, т.е. уделяя основное внимание требованиям к информации и бизнес-процессам, которые будет поддерживать будущая база данных.

Далее при помощи ERwin разработаем логическую модель данных, и создадим на ее основе физическую модель.

6.2. Разработка логической модели данных

Логическая модель описывает понятия предметной области, их взаимосвязь, а также ограничения на данные, налагаемые предметной областью. Логическая модель данных является визуальным графическим представлением структур данных, их атрибутов и связей. Логическая модель представляет данные таким образом, чтобы они легко воспринимались бизнес-пользователями.

Логическая модель данных является начальным прототипом будущей базы данных. Она строится в терминах информационных единиц, но без привязки к конкретной СУБД.

Основными компонентами логической модели являются:

- сущности;
- атрибуты сущности;
- связи между сущностями.

Сущность моделирует структуру однотипных информационных объектов (документов, хранилищ данных, таблиц базы данных). В пределах модели данных сущность имеет уникальное имя, выраженное существительным.

Сущность является шаблоном, на основании которого создаются конкретные экземпляры сущности.

Сущность обладает следующими свойствами:

- каждая сущность имеет уникальное имя, и к одному и тому же имени должна применяться одинаковая интерпретация;
- сущность обладает одним или несколькими атрибутами, которые либо принадлежат сущности либо наследуются через связь;
- сущность обладает одним или несколькими атрибутами, которые однозначно идентифицируют каждый экземпляр сущности;
- каждая сущность может обладать любым количеством связей с другими сущностями модели.

Атрибут – любая характеристика сущности, значимая для рассматриваемой предметной области и предназначенная для квалификации, идентификации, классификации, количественной характеристики или выражения состояния сущности. Атрибут представляет тип характеристик или свойств, ассоциированных со множеством реальных или абстрактных объектов (людей, мест, событий, состояний, идей, пар предметов и т.д.). Экземпляр атрибута – это определенная характеристика отдельного элемента множества. Экземпляр атрибута определяется типом характеристики и ее значением, называемым значением атрибута. В ER-модели атрибуты ассоциируются с конкретными сущностями. Таким образом, экземпляр сущности должен обладать единственным определенным значением для ассоциированного атрибута.

Связь – поименованная ассоциация между двумя сущностями, значимая для рассматриваемой предметной области. Связь – это ассоциация между сущностями, при которой, как правило, каждый экземпляр одной сущности, называемой родительской сущностью, ассоциирован с произвольным (в том числе нулевым) количеством экземпляров второй сущности, называемой сущностью-потомком, а каждый экземпляр сущности-потомка ассоциирован в точности с одним экземпляром сущности-родителя.

На примере бизнес-процесса управления продажами книг произведем идентификацию сущностей ПС и связей между ними, приведенных в табл. 7.

Таблица 7. Идентификация сущностей (пример заполнения)

Документ	Сущность
Накладные	Накладные
Счет-фактура	Счета
Книги	Товары
Клиенты	Клиенты
Заказы	Заказы

Исходя из данных табл. 7, каждой сущности в соответствие ставится документ. На данном этапе получаем пять основных сущностей, которые далее, при разработке физической модели данных, разобьем на подсущности.

Далее в табл. 8 приведем имена, атрибуты и назначения сущностей. Сущности в табл. 8 разделим на оперативные и справочные. Справочные сущности – это сущности, информация о которых хранится в справочниках. Как правило, это неизменная и многократно используемая в течение длительного периода времени информация. Информация об оперативных сущностях может меняться для каждого случая, как по назначению, так и по количеству. При разработке модели определим сущности, их первичные и внешние ключи и атрибуты, а также связи между сущностями.

Таблица 8. Сущности логической модели данных

Сущность	Атрибуты	Описание
Справочные		
1 Товары	<u>Код товара</u> , название книги, жанр, страницы, год издания, код автора, фамилия, имя, отчество, код издательства, название издательства, город сокращенно, город полностью	Информация о книгах авторов и издательствах
2 Виды накладных	<u>Код вида</u> , вид накладной	Информация о видах накладных
3 Клиенты	<u>Код клиента</u> , Фамилия, Имя, Отчество, телефон	Информация о клиентах
Оперативные		
4 Заказы	<u>Код заказа</u> , <i>Код товара</i> , количество, сумма, код поставщика, имя поставщика, номер телефона, адрес	Информация по заказам и поставщикам
5 Накладные	<u>Код накладной</u> , номер накладной, дата создания, вид накладной, цена, сумма, <i>код счета</i> , <i>код заказа</i> , <i>код клиента</i> , <i>код товара</i> , код сотрудника, фамилия, имя, отчество	Информация о накладных и сотрудниках
6 Счета	<u>Код счет-фактуры</u> , дата создания, <i>код заказа</i> , <i>код товара</i> , <i>код клиента</i> , количество, цена	Информация о счетах

При помощи CASE-средства ERWin разработаем ER-диаграмму логической модели данных на основе табл. 8, представленную на рисунке 10.

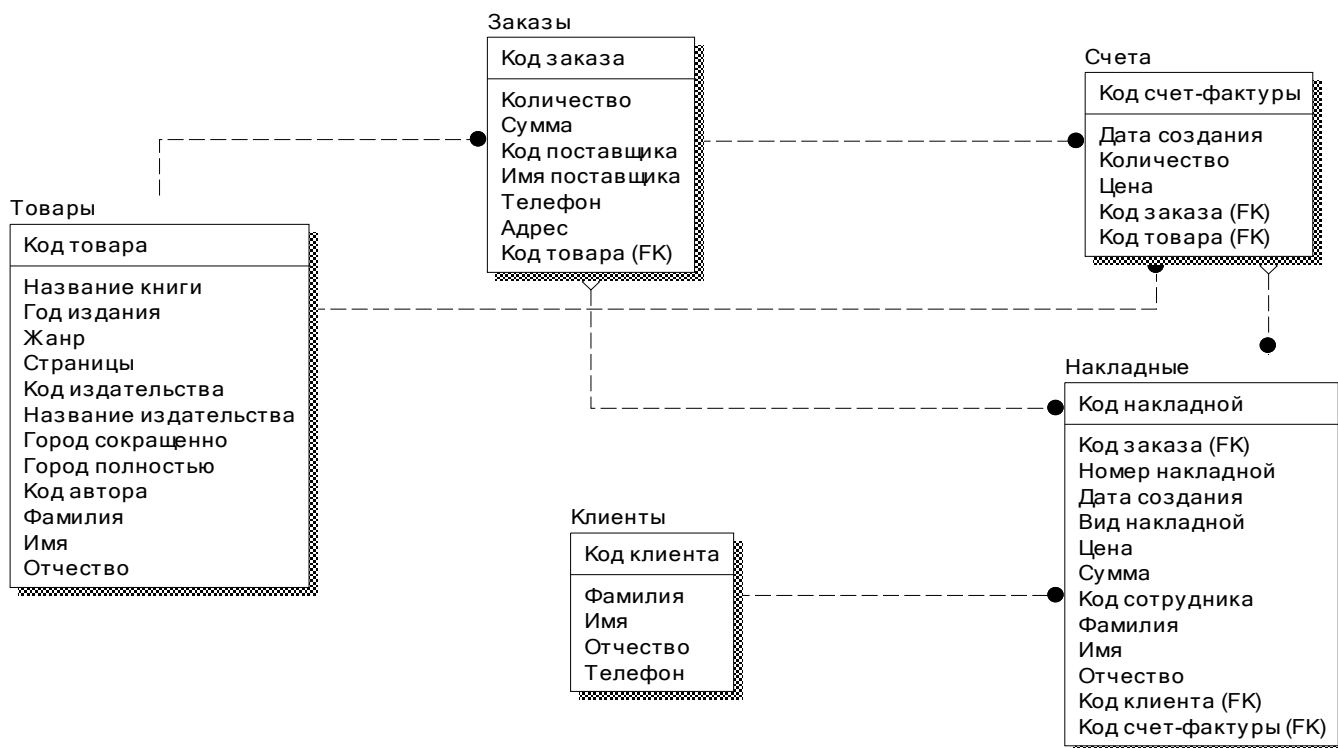
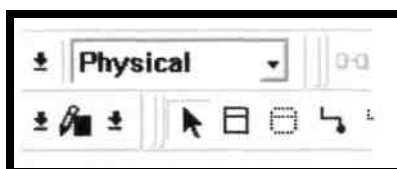


Рис. 10. ER-диаграмма логической модели данных

6.3. Разработка физической модели данных

Физическая модель баз данных определяет способы размещения данных в среде хранения и способы доступа к этим данным, которые поддерживаются на физическом уровне, является графическим представлением реально реализованной базы данных. В физической модели содержится информация обо всех объектах БД. Поскольку стандартов на объекты БД не существует (например, нет стандарта на типы данных), физическая модель зависит от конкретной реализации СУБД. Физическая база данных будет состоять из таблиц, столбцов и связей. Физическая модель зависит от платформы, выбранной для реализации, и требований к использованию данных. Компонентами физической модели данных являются таблицы, столбцы и отношения. Сущности логической модели, вероятно, станут таблицами в физической модели. Логические атрибуты станут столбцами. Логические отношения станут ограничениями целостности связей.

На основе логической модели данных разработаем при помощи CASE-средства ERWin ER-диаграмму физической модели данных. В ERWin необходимо выбрать тип модели:



Для этого выделим из сущностей подсущности для нормализации отношений. Отношения, разработанные на стадии формирования логической модели данных, преобразуются в таблицы, атрибуты становятся столбцами таблиц,

для ключевых атрибутов создаются уникальные индексы, домены преобразуются в типы данных, принятые в конкретной СУБД.

Ограничения, имеющиеся в логической модели данных, реализуются различными средствами СУБД, например, при помощи индексов, декларативных ограничений целостности, триггеров, хранимых процедур. При этом опять-таки решения, принятые на уровне логического моделирования определяют некоторые границы, в пределах которых можно развивать физическую модель данных. Точно также, в пределах этих границ можно принимать различные решения. Например, отношения, содержащиеся в логической модели данных, должны быть преобразованы в таблицы, но для каждой таблицы можно дополнительно объявить различные индексы, повышающие скорость обращения к данным. Многое тут зависит от конкретной СУБД.

Из сущности Товары выделяем подсущности Авторы и Издательства; из сущности Заказы выделяем Поставщиков и Позиции заказа; из сущности Накладные выделяем Позиции, Виды накладных и Сотрудников; из сущности Счета выделяем Позиции счета; сущность Клиенты оставляем неизменной (**рис. 12**).

Средствами ERWin выполним генерацию SQL-кода для создания реляционной базы данных ПС. Алгоритм действий следующий:

- При помощи CASE-средства ER-Win открыть физическую модель и вызвать диалог генерации SQL-кода (меню Tools → Forward Engineer → Schema Generation...).
- Выбрать формат будущей БД (SQL Server, Interbase, Oracle, и т.д.). Лучше отключить генерацию триггеров, для чего в группе триггер снимать все птички.
- Нажать на кнопку Preview....
- В появившемся диалоговом окне скопировать сгенерированный код в буфер обмена.

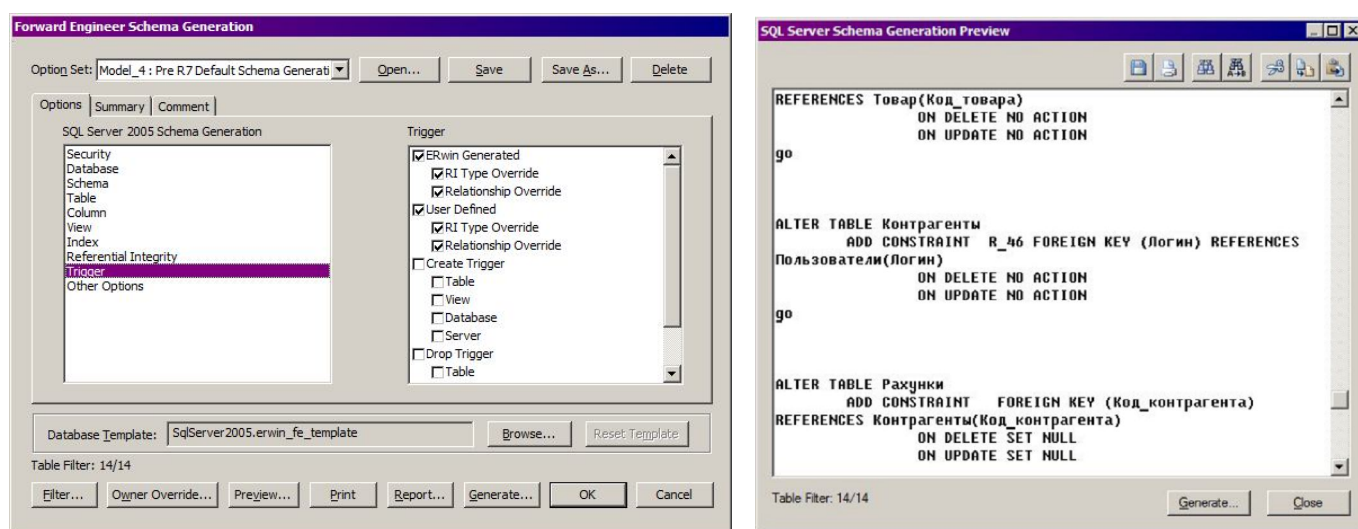


Рис. 11. Настройка и результаты процедуры генерации кода SQL

Формируется список разработанных таблиц и связей между таблицами БД, описание сущностей физической модели в формате табл. 8.

Задание

1. На основе модели классов UML, разработанной при выполнении [лабораторной работы №5](#), произвести идентификацию сущностей информационной базы ПС и связей между ними.
2. При помощи CASE-средства ERWin разработать ER-диаграмму логической модели данных.
3. В соответствии с требованиями технического задания, разработанного при выполнении лабораторной работы №3, провести обоснованный выбор СУБД.
4. Провести нормализацию сущностей логической модели данных и разработать ER-диаграмму физической модели данных **с учетом выбранной СУБД**.
5. Средствами ERWin на основе физической модели данных выполнить генерацию SQL-кода для создания реляционной базы данных ПС.
6. В выбранной СУБД развернуть БД, доработать её структуру с учетом возможной нормализации отношений, а также доработать структуру таблиц с учетом ограничений на значения полей. **В отчете представить изображение, подтверждающее факт развертывания БД.**
7. Выполнить описание таблиц БД в формате табл. 9 с указанием для всех полей типов данных выбранной СУБД. Столбцы «Условие на значение», «Значение по умолчанию» и «Примечание» заполнять при необходимости. Выполнить краткое описание таблиц БД и связей между ними.

Таблица 9. Структура таблицы

Имя поля	Тип данных	Размер, байт	Условие на значение*	Значение по умолчанию*	Примечание*
----------	------------	--------------	----------------------	------------------------	-------------

* Указанные столбцы не являются обязательными для заполнения

Требования к содержанию раздела

В подразделе 1 приводится анализ классов-сущностей из модели классов UML на предмет целесообразности хранения соответствующих данных в ПС и достаточности атрибутов этих классов для реализации автоматизированных функций ПС. Количество сущностей в модели зависит от предметной области. Классифицировать сущности на такие, которые отражают:

- а) справочные, отражающие нормативно-справочную (условно-постоянную) информацию о предметной области;
- б) оперативные, отражающие входную (текущую) информацию для решения текущих задач процесса информатизации.

В подразделе 2 строится логическая модель данных. Сущности логической модели данных следует разделить на оперативные и справочные. Имена, атрибуты и назначение сущностей логической модели данных привести в формате табл. 8. При разработке модели определить сущности, их первичные и внешние ключи и атрибуты, а также связи между сущностями.

Для моделирования данных использовать CASE-средство ERWin либо побочное ему инструментальное средство.

В подразделе 3 сущности логической модели данных рассматриваются как реляционные отношения и выполняется нормализация этих отношений. Таким образом осуществляется физическое моделирование данных, цель которого – таблицы в нормальных формах высшего, минимум, третьего (ЗНФ) порядка. При проведении нормализации следует стремиться к исключению аномалий модификации данных, к атомарности атрибутов, в первую очередь первичных ключей, и к минимизации дублирования данных.

По результатам физического моделирования данных осуществляется генерацию SQL-кода для создания реляционной базы данных ПС. Полученный SQL-код приводится в приложении к отчету.

В подразделе 3 отражается обоснованный выбор СУБД посредством сравнения двух-трех современных СУБД примерно равных по своим функциональным возможностям. Выбор СУБД должен быть обоснован масштабом проекта и особенностями решаемых в нем задач.

В этом же подразделе отражается развертывание БД на базе выбранной СУБД. **Необходимо представить доказательный результат развертывания, полученный средствами СУБД**, и выполнить его краткое описание. Также следует представить краткое описание таблиц БД в формате табл. 10 и описание всех связей между таблицами в формате табл. 11. Структуру всех таблиц БД в формате табл. 9, вынести в приложение к отчету.

Таблица 10. Список разработанных таблиц

№ п/п	Имя таблицы	Описание
----------	-------------	----------

Таблица 11. Связи между таблицами БД

Родительская таблица		Дочерняя таблица		Тип связи*
Название	Атрибут РК	Название	Атрибут FK	

* в качестве типа связи указывать «1:1», «1:M», «M:M»

Контрольные вопросы и упражнения

1. Что такое информационное обеспечение ПС?
2. Что такое информационная база ПС?
3. Опишите алгоритм создания информационной базы ПС?
4. Какие современные СУБД используются для создания информационной базы ПС?
5. Что такое нормализация отношений БД?
6. Чем отличается физическая модель данных от логической?
7. В чем различия между справочными таблицами и таблицами оперативных данных?

Лабораторная работа №7

Тема: Разработка программного обеспечения ПС

Цель: Приобретение навыков проектирования и разработки программной системы при помощи современных инструментальных средств.

Теоретические положения

7.1. Классификация ПО

Программное обеспечение (ПО) ПС – это комплекс компьютерных программ, обеспечивающих обработку и передачу данных, а также документация по их конфигурированию и применению. Включает в себя системное, служебное, инструментальное, прикладное и специальное ПО.

Системное ПО – это операционные системы, управляющие функционированием средств вычислительной техники, сетевого оборудования и прикладного ПО.

Служебное ПО – это программные средства, выполняющие функции по обслуживанию компьютерной техники и программ других классов: различные утилиты, средства диагностики, антивирусные программы, и т.п.

Инструментальное ПО – это совокупность программных средств, необходимых для проектирования и разработки ПС: CASE, СУБД, интегрированные среды разработки (IDE), интерпретаторы и трансляторы программ, и т.п.

Прикладное ПО – это совокупность программ, необходимых для обеспечения функционирования программ решения задач управления объектом: внешние библиотеки для прикладных программ, средства защиты данных, офисное ПО, графические пакеты, браузеры, а также иные программы необходимые для полноценного использования специального ПО.

При проектировании ПО проводится анализ и обоснованный выбор инструментального ПО (СУБД, IDE, CASE-средства и др.). Для анализа выбирается не менее 2-х современных систем (языков) программирования или инструментальных средств разработки (IDE), которые пригодны для разработки соответствующих программ. Для каждого средства указываются основные возможности и характерные особенности без подробного описания языка программирования, интегрированной среды и т.п. В результате анализа делается вывод, на основании чего выбирается средство разработки специального ПО.

По результатам разработки общесистемного ПО строится его структурная схема в виде древовидной блок-схемы либо организационной диаграммы (рис. 13).

К программному обеспечению, подлежащему разработке в процессе рабочего проектирования, относится категория специального ПО (СПО). СПО – это совокупность программ, непосредственно реализующих алгоритмы решения функциональных задач. СПО разрабатывается при проектировании ПС на базе математического обеспечения и представляет собой программную реализацию моделей, разработанных в при выполнении лабораторной работы №4 и 4.1. Зачастую включает в себя интерфейсные средства: web-страницы, формы, отчеты,

модули кода для реализации серверной части СПО, а также запросы, представления, курсоры, хранимые процедуры, макросы и другие модули программного кода. По результатам разработки специального ПО строится его структурная схема в виде диаграммы компонентов UML, либо в произвольной нотации.

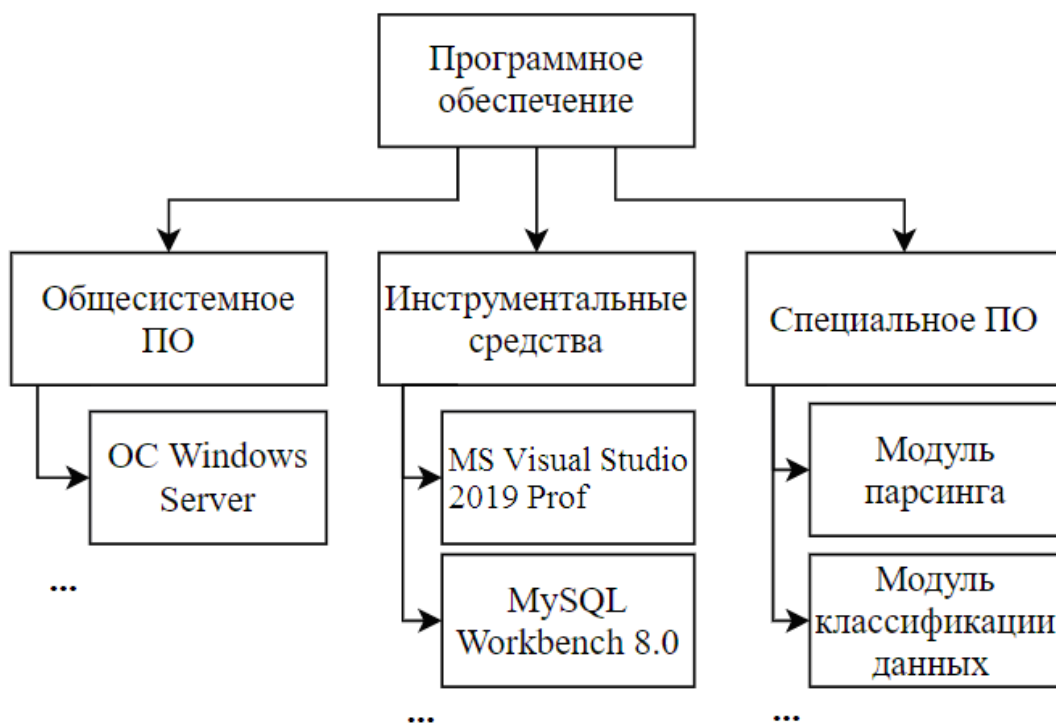


Рис. 13. Структурная схема общесистемного программного обеспечения (пример)

7.2. Диаграммы компонентов UML

Для визуализации статического аспекта физических компонентов ПО и их отношений, а также для специфицирования деталей этих компонентов в UML используются диаграммы компонентов (component diagram). Диаграмма компонентов позволяет определить состав программных компонентов, в роли которых может выступать исходный, бинарный и исполняемый код, и др., а также установить зависимости между ними.

При разработке диаграмм компонентов преследуются такие цели:

- спецификация общей структуры исходного кода системы;
- спецификация исполнимого варианта системы.

Для графического представления компонентов используются прямоугольники. Внутри прямоугольника записывается имя компонента и стереотип. Возможные типы компонентов показаны в табл. 12. Компоненты-файлы на схеме приводятся с указанием расширения.

При спецификации общей структуры исходного кода ПС необходимо учитывать специфику используемого языка программирования. В частности в Java рекомендуется отдельный класс описывать в отдельном файле, несмотря на то, что язык позволяет описывать несколько классов в одном файле и использовать механизм внутренних классов. Диаграмма компонентов, применяемая для рассматриваемой цели, изображена на рис. 14.

Таблица 12. Типы компонентов приложения

Стереотип	Тип компонента
«source»	файл исходного текста программы
«form»	файл интерфейсной формы или web-страницы
«executable»	исполняемый файл
«library»	статическая или динамическая библиотека
«document»	файл документации
«data»	файл данных
«table»	таблица базы данных
«view»	представление как объект базы данных
«file»	любой другой файл, кроме указанных выше в этой таблице

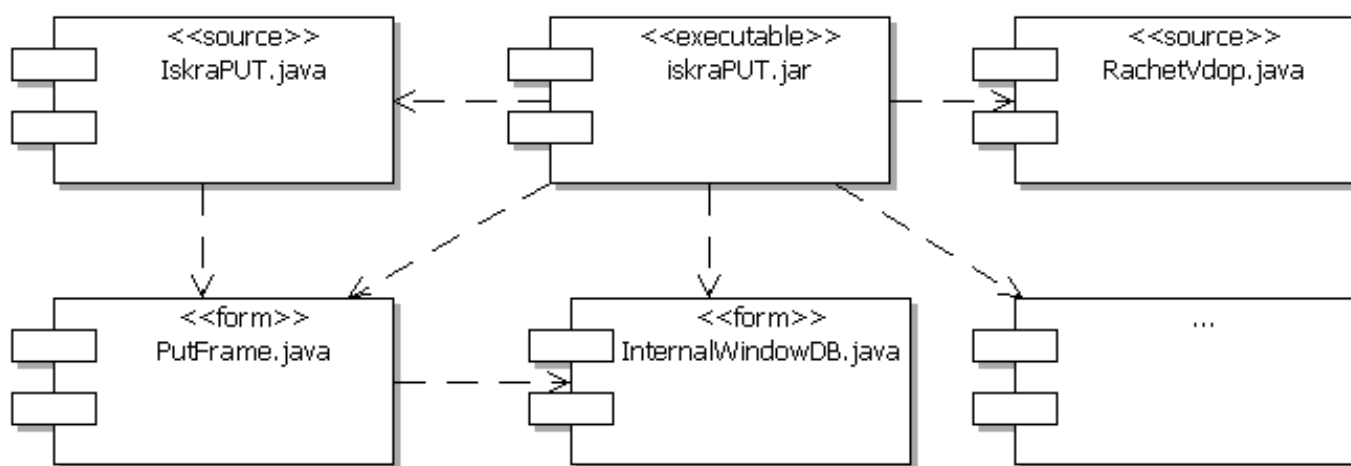


Рис. 14. Фрагмент диаграммы компонентов, специфицирующей структуру исходного кода

Диаграмма компонентов, отражающая взаимодействие компонентов разных типов нагляднее представляются с использованием *стереотипных* элементов (пиктограмм). На рис. 15. показан пример такой модели со стереотипными элементами типа «source», «library», «form», «document», «table».

7.3. Диаграммы развертывания UML

Диаграмма развертывания, или применения (deployment diagram), – это еще один из двух видов диаграмм, используемых при моделировании физических аспектов объектно-ориентированной системы. Такая диаграмма показывает конфигурацию узлов, где производится обработка информации, и то, какие компоненты размещены на каждом узле. Диаграммы развертывания используются для моделирования статического вида системы с точки зрения развертывания. В основном под этим понимается моделирование топологии аппаратных средств, на которых выполняется система. Диаграммы развертывания важны для визуализации, специфицирования и документирования встроенных, клиент-серверных и распределенных систем.

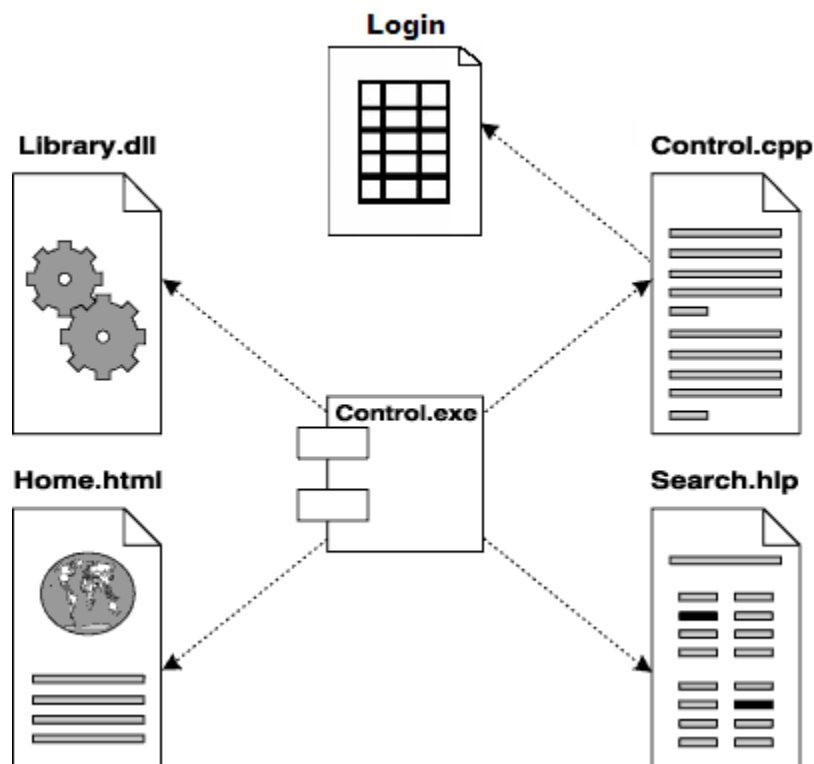


Рис. 15. Зависимость между компонентами приложения с использованием стереотипных элементов

Для графического представления элементов диаграмм развертывания используются параллелепипеды. Внутри параллелепипеда записывается имя и стереотип. Возможные типы узлов показаны в табл. 13.

Таблица 13. Типы элементов диаграмм развертывания UML

Стереотип	Тип элемента
«processor»	Ресурсоемкий узел
«device»	Нересурсоемкий узел (устройство)
«sensor»	Измерительное устройство (датчик)
«printer»	Печатающее устройство (принтер)
«net»	Сетевая передающая среда
«database»	База данных
«node»	Узел, не попадающий под тип, указанный выше в этой таблице

На рис. 16 показан фрагмент соединения типа «Клиент-Сервер» посредством локальной вычислительной сети.

Кроме соединений на диаграмме развертывания могут присутствовать отношения зависимости между узлом и размещаемыми на нем компонентами. Подобный способ является альтернативой вложенному изображению компонентов внутри символа узла, что не всегда удобно, поскольку делает этот символ излишне объемным. Поэтому при большом количестве развернутых на узле компонентов соответствующую информацию можно представить в форме отношения зависимости, которая обозначается пунктирной линией (рис. 17).

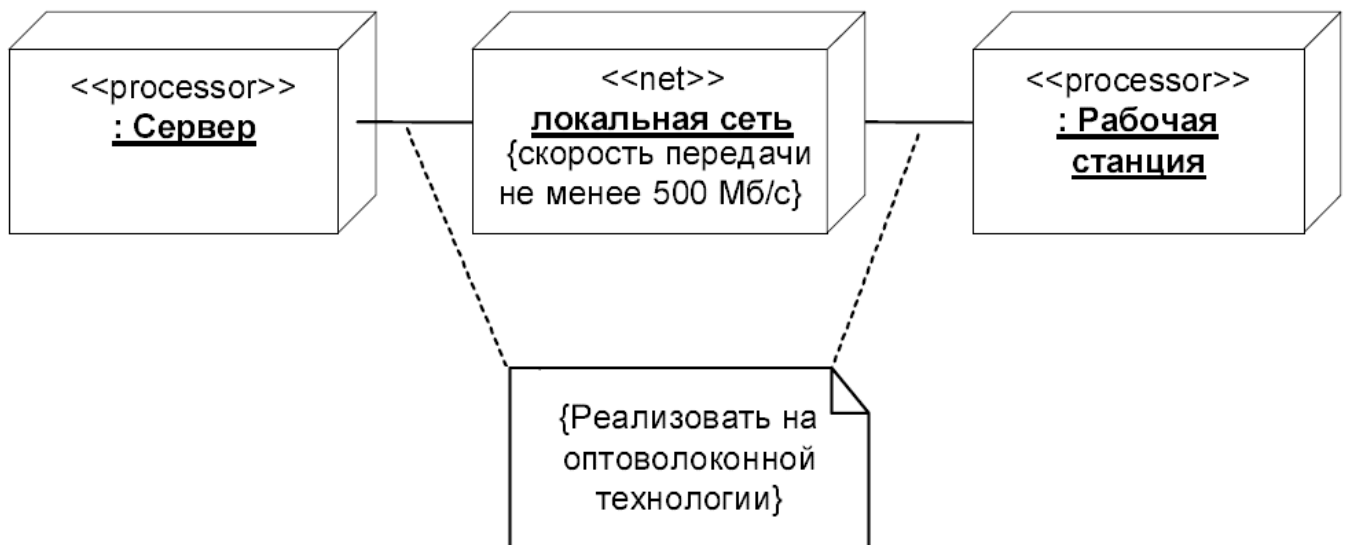


Рис. 16. Фрагмент диаграммы развертывания с соединениями между узлами

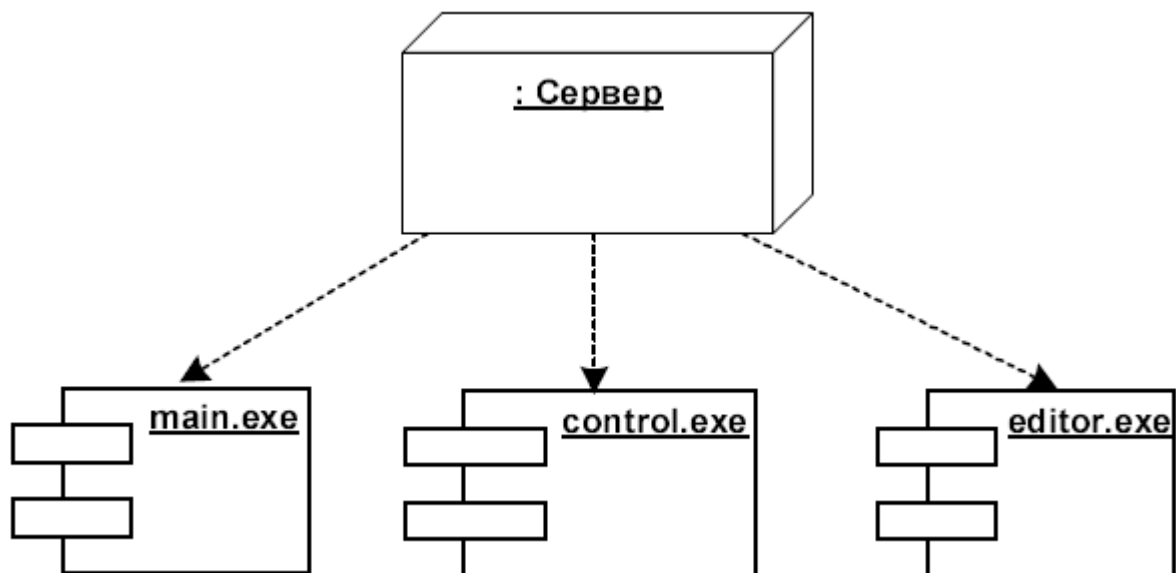


Рис. 17. Диаграмма развертывания с отношением зависимости между узлом и развернутыми на нем компонентами

Задание

1. В соответствии с требованиями технического задания, разработанного при выполнении лабораторной работы №3, провести обоснованный выбор средства разработки специального ПО. Разработать схему общесистемного ПО на подобие схемы, показанной на рис. 13.
2. В соответствии с требованиями технического задания, разработанного при выполнении лабораторной работы №3, а также проектными решениями, разработанными при выполнении лабораторных работ №4 и №6, разработать специальное программное обеспечение ПС.

3. Выполнить описание разработанных компонентов приложения в виде табл. 14. Типы компонентов указать согласно табл. 12. Имена компонентов-файлов привести с указанием расширения.

Таблица 14. Перечень разработанных компонентов приложения

№	Имя	Тип	Описание
---	-----	-----	----------

4. Построить структурную схему разработанного приложения в виде диаграммы компонентов UML, выражающую взаимодействие его компонентов с компонентами БД в процессе функционирования приложения.
5. Запустить приложение на выполнение. Убедиться в соответствии результатов выполнения приложения требованиям, установленным в техническом задании. При обнаружении логических ошибок задокументировать их и устранить.
6. Представить экранные формы компонентов приложения, в том числе отчетов.
7. Проанализировать код приложения по критерию сложности. В качестве критерия сложности использовать:
- число модулей (классов) приложения;
 - суммарное число переменных подпрограмм (методов классов), включая их формальные параметры;
 - суммарное количество операторов подпрограмм (методов классов);
 - глубину вложенности структурных операторов ветвления и повторения;
 - глубину наследования классов.
8. Выполнить описание физических элементов ПС в виде табл. 15. Типы элементов указать согласно табл. 13.

Таблица 15. Перечень узлов программной системы

№	Имя	Тип	Описание
---	-----	-----	----------

9. Построить диаграмму развертывания UML, выражающую зависимости между узлами ПС и развернутыми на них компонентами из табл. 14.

Требования к содержанию отчета

В подразделе 1 проводится анализ и обоснованный выбор инструментальных средств для разработки специального ПО. Для анализа выбирается не менее 2-х современных систем (языков) программирования, которые пригодны для разработки программных компонентов ПС. Для каждого средства указываются основные возможности и характерные особенности без подробного описания языка программирования, интегрированной среды и т.п. В результате анализа делается вывод, на основании чего выбирается средство разработки специального ПО.

В подразделе 2 кратко документируется разработанное специальное ПО. Выполняется описание разработанных модулей и компонентов, как по тексту, так и в виде таблиц 14 и 15. Листинг кода может быть представлен в приложении.

В подразделе 3 представляется диаграмма компонентов и выполняется её краткое описание. В описании должны раскрываться существенные детали реализации и взаимодействия компонентов (например, технологии реализации или обмен данными и др.). При этом следует избегать использования стандартного стереотипа «file», а максимально дифференцировать компоненты по типам согласно табл. 12.

В подразделе 4 представляется диаграмма развертывания и выполняется её краткое описание. В описании должны раскрываться существенные детали развертывания и взаимодействия узлов (например, важные с точки зрения понимания схемы технические характеристики, обмен данными между узлами, отношения узлов с компонентами и др.). При этом следует избегать использования стандартного стереотипа «node», а максимально дифференцировать компоненты по типам согласно табл. 13. Также следует различать на диаграмме узлы и реализованные на их базе компоненты, как это показано на рис. 17.

Контрольные вопросы и упражнения

1. Что такое программное обеспечение ПС?
2. Каковы отличия между сервисными программами и утилитами?
3. Какие типы программных средств относятся к специальному ПО.
4. Как происходит разработка программного кода ПС?
5. Что такое компиляция и построение программы?
6. Как в интегрированной среде программирования осуществляется обнаружение синтаксических ошибок?
7. Какова связь компонентов ПО с компонентами ИО разрабатываемой ПС?
8. Что такое специальное ПО?
9. Какие бывают типы компонентов специального ПО?
10. Каково назначение диаграммы компонентов UML?
11. Какие бывают типы физических элементов ПС?
12. Каково назначение диаграммы развертывания UML?
13. Определить сложность заданного фрагмента программного кода.
14. Предложить способы оценивания производительности и эффективности программного обеспечения на базе данного кода.

Лабораторная работа №8

Тема: Тестирование ПС

Цель: Освоение методики сценарного и интеграционного тестирования ПО, в том числе разработанной программной системы в ручном режиме.

Теоретические положения

8.1. Тестирование, как способ контроля качества ПО.

Тестирование – это проверка соответствия между реальным и ожидаемым поведением программы в специально заданных условиях. Исходной информацией для тестирования является сведения о том, как программная система должна себя вести, то есть требования к ней или к ее отдельной части. Наиболее распространенным способом тестирования программ является метод *черного ящика*. При этом реализация системы недоступна тестирующим, и тестируется только ее интерфейс посредством пользовательского воздействия. Противоположностью данному методу является тестирование методом *белого ящика*, когда исходный код доступен тестирующим и используется в качестве источника информации о тестируемой системе. В этом случае на основе требований к системе помимо ее реализации создается (программируется) ее тестовая модель.

По способу реализации тесты могут быть ручными и автоматическими.

Ручной тест – это последовательность действий тестирующего (или разработчика), которую он может воспроизвести и при этом ошибка произойдет. Как правило, в средствах контроля ошибок такие последовательности действий содержатся в некоторой форме описания ошибки.

Автоматический тест – это некоторая программа, которая воздействует на систему и проверяет то или иное ее свойство (функциональное требование).

8.2. Критерии тестирования

В любом случае качество тестирования может оказаться неудовлетворительным – ошибки находятся редко или вообще не находятся ввиду того, что количество возможных состояний программы очень велико, и тестирование не в состоянии покрыть их все.

На практике в реальных проектах, определяют критерии тестирования, которые определяют тот уровень качества, который необходимо обеспечить в данном проекте. Хорошее качество стоит дорого и очевидно, что разное ПО имеет разное качество, например, система управления химическим реактором и текстовый редактор. В силу ограниченности ресурсов на тестирование также целесообразно определить те аспекты ПО, которые наиболее важны как для общей работоспособности системы, так и для удовлетворения заказчика.

При тестировании на некоторых (но не на всех возможных) входных данных применяют принцип факторизации – множество всех возможных входных значений разбивается на значимые с точки зрения тестирования классы и "прогоняются" тесты не на всех возможных входных значениях, но берут по одному набору значений из каждого класса. Например, тестируют некоторую функцию системы на ее граничные значения – очень большие значения параметров, очень маленькие и т.п. Часто факторизацию удобно делать, исходя из требований к данной функции. Также бывает полезно посмотреть на ее реализацию и протестировать ее по разным логическим веткам (порождаемым, например, условными операторами).

8.3. Виды тестирования

Модульное тестирование – тестируется отдельный модуль, в отрыве от остальной системы. Самый распространенный случай применения – тестирования модуля самим разработчиком, проверка того, что отдельные модули, классы, методы выполняют действительно то, что от них ожидается. Различные среды разработки широко поддерживают средства модульного тестирования. Созданные разработчиком модульные тесты часто включаются в пакет регрессионных тестов и таким образом, могут запускаться многократно.

Интеграционное тестирование – два и более компонентов тестируются на совместимость. Это очень важный вид тестирования, поскольку разные компоненты могут создаваться разными людьми, в разное время, на разных технологиях. Этот вид тестирования должен применяться самими программистами, чтобы, по крайней мере, удостовериться, что все компоненты программы совместимы в первом приближении. Далее тонкости интеграции могут исследовать тестировщики.

Необходимо отметить, что такого рода "ошибки на стыках" непросто обнаруживать и устранять. Во время разработки все компоненты все вместе не готовы, интеграция откладывается, а в конце обнаруживаются трудные ошибки (в том смысле, что их устранение требует существенной работы: реструктуризации, рефакторинга). Здесь выходом является ранняя интеграция системы и в дальнейшем использование практики постоянной интеграции.

Сценарное тестирование – системный подход к тестированию ПО по предварительно написанным и задокументированным тест-кейсам (сценариям) – последовательностей действий пользователя в приложении.

Системное тестирование – это тестирование всей системы в целом, как правило, через ее пользовательский интерфейс. При этом могут использоваться как ручные, так и автоматические тесты с акцентом на том, как ПО выглядит и работает в целом, удобно ли оно, удовлетворяет ли она ожиданиям заказчика. При этом могут открываться различные дефекты, такие как неудобство в использовании тех или иных функций, забытые или "скудно" понятые требования.

Регрессионное тестирование – тестирование системы в процессе ее разработки и сопровождение на не регресс. То есть проверяется, что изменения системы не ухудшили уже существующей функциональности. Для этого создаются пакеты регрессионных тестов, которые запускаются с определенной

периодичностью – например, в пакетном режиме, связанные с процедурой постоянной интеграции.

Нагрузочное тестирование – тестирование системы на корректную работу с большими объемами данных. Например, проверка баз данных на корректную обработку большого (предельного) объема записей, исследование поведение серверного ПО при большом количестве клиентских соединений, эксперименты с предельным трафиком для сетевых и телекоммуникационных систем, одновременное открытие большого числа файлов, проектов и т.д.

Стрессовое тестирование – тестирование системы на устойчивость к непредвиденным ситуациям. Этот вид тестирования нужен далеко не для каждой системы, так как подразумевает высокую планку качества.

Приемочное тестирование – тестирование, выполняемое при приемке системы заказчиком. Более того, различные стандарты часто включают в себя наборы приемочных тестов.

8.4. Сценарное тестирование

Сценарное тестирование предполагает разделение задачи на этапы (подготовка, выполнение, завершение и т.д.) и последовательное выполнение этих этапов.

Особенности этого подхода:

- четкое понимание того, какие функции покрыты тестами;
- уверенность, что все задокументированные тест-кейсы будут пройдены в срок;
- быстрое и легкое подключение нового специалиста на проект благодаря наличию подробных сценариев.

Пример. При тестировании почты, сценарное тестирование может выглядеть так:

- перейти на стартовую страницу;
- нажать на кнопку «Новый»;
- в поле «Кому» написать j.cannegieter@squerist.nl;
- нажать на кнопку «Файл»;
- выбрать документ «Тест1»;
- в поле «Тема» написать «Тестовое вложение <дата>»;
- открыть почтовый ящик j.cannegieter;
- проверить, доставлено ли сообщение и вложение;

Для этого же примера сценарий общего (исследовательского) тестирования может выглядеть так:

- создать и отправить письмо с вложением одному получателю.
- проверить, доставлено ли сообщение и вложение.

8.5. Интеграционное тестирование

Интеграционное тестирование предназначено для проверки связи между компонентами, а также взаимодействия с различными частями системы (операционной системой, оборудованием либо связи между разными системами).

Уровни интеграционного тестирования:

1. Компонентный интеграционный уровень (*Component Integration testing*). Проверяется взаимодействие между компонентами системы после проведения компонентного (модульного) тестирования.

2. Системный интеграционный уровень (*System Integration Testing*), также известное как Тестирование взаимодействия (*Interoperability Testing*). Проверяется взаимодействие системы с другими системами после проведения системного тестирования.

Подходы к интеграционному тестированию:

1. Снизу вверх (*Bottom Up Integration*). Все низкоуровневые модули, процедуры или функции собираются воедино и затем тестируются. После чего собирается следующий уровень модулей для проведения интеграционного тестирования.

2. Сверху вниз (*Top Down Integration*). Вначале тестируются все высокоуровневые модули, и постепенно один за другим добавляются низкоуровневые. Все модули более низкого уровня симулируются заглушками с аналогичной функциональностью, затем по мере готовности они заменяются реальными активными компонентами.

3. Большой взрыв ("*Big Bang*" *Integration*). Все или практически все разработанные модули собираются вместе в виде законченной системы или ее основной части, и затем проводится интеграционное тестирование. Такой подход очень хорош для сохранения времени. Однако если тест кейсы и их результаты записаны не верно, то сам процесс интеграции сильно осложнится, что станет преградой для команды тестирования при достижении основной цели интеграционного тестирования

Например. Установлено, что данные по продажам из портала передаются в аналитическую систему. Тогда результатом интеграционного тестирования является решение о том, что необходимо дополнительно проверить, что:

- а) продажи отправлены в корректном формате;
- б) данные по продажам получены аналитической системой в корректном формате;
- в) потери данных не произошло.

Задание

Выполнить системное пользовательское тестирование работоспособности ПС, разработанную при выполнении [лабораторной работы №7](#), посредством воздействия на её интерфейсную часть. Для этого необходимо:

1. Согласно функциональным требованиям к ПС, предъявленным при выполнении [лабораторной работы №3](#), разработать несколько сценариев тестирования ПС.

2. Реализовать сценарии и задокументировать полученные результаты тестирования.

3. По согласованию с преподавателем разработать сценарий тестирования стороннего программного продукта согласно данным, представленным в [приложении Д](#). Оформить отчет по выполненному сценарному тестированию.

4. Выполнить интеграционное тестирование на компонентном уровне разработанных при выполнении [лабораторной работы №7](#) автоматизированных функций программного продукта, используя подход *Bottom Up Integration*.

5. Выполнить нагрузочное тестирование программы и оценить эффективность разработанных при выполнении [лабораторной работы №7](#) автоматизированных функций ПС, запросов к БД. В качестве критерия эффективности использовать время выполнения функции. Предварительно подготовить 5 массивов исходных данных на 10, 50, 100, 500 и 1000 записей. Построить графики зависимости времени вычислений от объема исходных данных.

Требования к содержанию отчета.

В подразделе 1 приводятся все разработанные сценарии для тестирования ПС, а также результаты сценарного тестирования посредством проверки работоспособности программы. В идеале разработанные сценарии должны соответствовать автоматизированным функциям ПС и инициировать срабатывание всех команд меню, ссылок и других пользовательских инструментов, предусмотренных требованиями.

В подразделе 2 документируются обнаруженные логические ошибки (при наличии таковых). Для каждой логической ошибки:

- описать ее сущность;
- представить тестовую последовательность манипуляций для ее обнаружения (как правило, это ссылка на сценарий и подмножество шагов полной тестовой последовательности в сценарии);
- представить форму ее проявления в результате применения теста;
- определить форму ее выражения в коде;
- описать способ ее устранения.

Подраздел 3 формируется в случае выдачи преподавателем дополнительного задания из [приложения Д](#). Необходимо отразить сценарное тестирование некоторого интернет-магазина. Для заданных задач для сценарного тестирования представить разработанные сценарии и результаты их реализации аналогично содержанию подразделов 1 и 2. В силу специфики тестируемого продукта тестирование проводить методом черного ящика. Поэтому форму выражения ошибки в коде и способ её устранения в данном случае приводить не нужно.

В подразделе 4 необходимо описать сценарии тестирования работоспособности отдельных модулей программного продукта. Выполнить эти сценарии. Затем выполнить аналогичное тестирование на более высоком уровне, объединяющем работу нескольких модулей.

Наивысший уровень – полный цикл работы всего программного продукта, проверка которого осуществляется посредством системного тестирования.

Результаты интеграционного тестирования на всех уровнях представить аналогично подразделу 2.

В подразделе 5 необходимо представить технические характеристики машины, используемой для анализа производительности программы. Привести определенную последовательность действий пользователя (наподобие последовательности проверочных действий в сценарии, как в подразделе 1). Привести временные оценки проверок для каждой конфигурации и кратко их прокомментировать. Привести временные оценки нагрузочных проверок в виде сводной таблицы и графика на каждом файле для каждого метода. Прокомментировать результаты выполненного анализа.

Контрольные вопросы и упражнения

1. Перечислите и охарактеризуйте методы обеспечения качества программного обеспечения.
2. Что такое тестирование компьютерной программы?
3. Что такое ожидаемое поведение программы?
4. Какие виды тестов и тестирования программного обеспечения применимы к разработанному программному приложению?
5. Что такое производительность программного обеспечения?
6. Укажите методы и способы оценивания производительности программного обеспечения.
7. В чем заключается идентификация ошибок в программе?
8. Какими способами можно обнаружить логически ошибки в коде?
9. Перечислите показатели, по которым выполняется анализ кода.
10. Что понимается под эффективностью программного обеспечения?
11. Что понимается под сложностью программного обеспечения?
12. Каковы могут быть формы представления результатов оценивания эффективности программного обеспечения?
13. От чего зависит эффективность программного обеспечения?
14. Что такое модульное (компонентное) тестирование?
15. Укажите цели разработки сценариев тестирования.
16. В чем разница между тест кейсом и тестовым сценарием?
17. Перечислите преимущества использования сценариев при тестировании программных продуктов.

Лабораторная работа №9

Тема: Разработка баг-репорта ПС

Цель: Приобретение навыков выявления дефектов (багов), определения их характеристик, построение баг-репорта.

Теоретические положения

9.1. Структура баг-репорта

Баг-репорт (*bug report*) – это технический документ, описывающий ситуацию или последовательность действий, приведшую к некорректной работе объекта тестирования, с указанием причин и ожидаемого результата. Шаблон баг-репорта приведен на рис. 18. Его составляет тестировщик, чтобы разработчикам было понятно, что работает неправильно, насколько дефект критичен и что нужно исправить. Баг-репорты – часть программного процесса.

Шапка	
Короткое описание (Summary)	Короткое описание проблемы, явно указывающее на причину и тип ошибочной ситуации.
Проект (Project)	Название тестируемого проекта
Компонент приложения (Component)	Название части или функции тестируемого продукта
Номер версии (Version)	Версия на которой была найдена ошибка
Серьезность (Severity)	Наиболее распространена пятиуровневая система градации серьезности дефекта: • S1 Блокирующий (Blocker) • S2 Критический (Critical) • S3 Значительный (Major) • S4 Незначительный (Minor) • S5 Тривиальный (Trivial)
Приоритет (Priority)	Приоритет дефекта: • P1 Высокий (High) • P2 Средний (Medium) • P3 Низкий (Low)
Статус (Status)	Статус бага. Зависит от используемой процедуры и жизненного цикла бага (bug workflow and life cycle)
Автор (Author)	Создатель баг репорта
Назначен на (Assigned To)	Имя сотрудника, назначенного на решение проблемы
Окружение	
ОС / Сервис Пак и т.д. / Браузера + версия / ...	Информация об окружении, на котором был найден баг: операционная система, сервис пак, для WEB-тестирования – имя и версия браузера и т.д.
...	
Описание	
Шаги воспроизведения (Steps to Reproduce)	Шаги, по которым можно легко воспроизвести ситуацию, приведшую к ошибке.
Фактический Результат (Result)	Результат, полученный после прохождения шагов к воспроизведению
Ожидаемый результат (Expected Result)	Ожидаемый правильный результат
Дополнения	
Прикрепленный файл (Attachment)	Файл с логами, скриншот или любой другой документ, который может помочь прояснить причину ошибки или указать на способ решения проблемы

Рис. 18. Шаблон баг-репорта

9.2. Основные характеристики и виды дефектов тестируемого ПО

Градация Серьезности дефекта (Severity)

Рассмотрим градации дефектов (багов) ПО.

S1 Блокирующая (Blocker). Блокирующая ошибка, приводящая приложение в нерабочее состояние, в результате которого дальнейшая работа с тестируемой системой или ее ключевыми функциями становится невозможна. Решение проблемы необходимо для дальнейшего функционирования системы.

S2 Критическая (Critical). Критическая ошибка, неправильно работающая ключевая бизнес-логика, дыра в системе безопасности, проблема, приведшая к временному падению сервера или приводящая в нерабочее состояние некоторую часть системы, без возможности решения проблемы, используя другие входные точки. Решение проблемы необходимо для дальнейшей работы с ключевыми функциями тестируемой системой.

S3 Значительная (Major). Значительная ошибка, часть основной бизнес-логики работает некорректно. Ошибка не критична или есть возможность для работы с тестируемой функцией, используя другие входные точки.

S4 Незначительная (Minor). Незначительная ошибка, не нарушающая бизнес-логику тестируемой части приложения, очевидная проблема пользовательского интерфейса.

S5 Тривиальная (Trivial). Тривиальная ошибка, не касающаяся бизнес-логики приложения, плохо воспроизводимая проблема, малозаметная посредством пользовательского интерфейса, проблема сторонних библиотек или сервисов, проблема, не оказывающая никакого влияния на общее качество продукта.

Градация Приоритета дефекта (Priority)

P1 Высокий (High). Ошибка должна быть исправлена как можно быстрее, т.к. ее наличие является критической для проекта.

P2 Средний (Medium). Ошибка должна быть исправлена, ее наличие не является критичной, но требует обязательного решения.

P3 Низкий (Low). Ошибка должна быть исправлена, ее наличие не является критичной, и не требует срочного решения.

Порядок исправления ошибок по их приоритетам: High -> Medium -> Low.

9.3. Требования к количеству открытых дефектов

Наличие открытых дефектов P1, P2 и S1, S2, считается неприемлемым для проекта. Все подобные ситуации требуют срочного решения и идут под контроль к менеджерам проекта. Наличие строго ограниченного количества открытых ошибок P3 и S3, S4, S5 не является критичным для проекта и допускается в выдаваемом приложении. Количество же открытых ошибок зависит от размера проекта и установленных критериев качества.

Все требования к открытым ошибкам оговариваются и документируются на этапе принятия решения о качестве разрабатываемого продукта.

Баг-репорт – это технический документ. Поэтому язык описания проблемы должен быть техническим. Должна использоваться правильная терминология при использовании названий элементов пользовательского интерфейса (editbox, listbox, combobox, link, text area, button, menu, popup menu, title bar, system tray и т.д.), действий пользователя (click link, press the button, select menu item и т.д.) и полученных результатах (window is opened, error message is displayed, system crashed и т.д.).

9.4. Требования к обязательным полям баг-репорта

Обязательными полями баг-репорта являются: короткое описание (Bug Summary), серьезность (Severity), шаги к воспроизведению (Steps to reproduce), результат (Actual Result), ожидаемый результат (Expected Result). Ниже приведены требования и примеры по заполнению этих полей.

Короткое описание. Название говорит само за себя. В одном предложении надо уместить смысл всего баг-репорта, а именно: коротко и ясно, используя правильную терминологию сказать, что и где не работает. Например:

1. Приложение зависает, при попытке сохранения текстового файла размером больше 50Мб.
2. Данные на форме "Профайл" не сохраняются после нажатия кнопки "Сохранить".

Также применяется [Принцип "Где? Что? Когда?"](#), который заключается в составлении предложения, в котором факты дефекта изложены в следующей последовательности:

- Где? В каком месте интерфейса пользователя или архитектуры программного продукта находится проблема. Начинать предложение следует с существительного, а не предлога.
- Что? Что происходит или не происходит согласно спецификации или представлению о нормальной работе программного продукта. При этом следует указывать на наличие или отсутствие объекта проблемы, а не на его содержание (его указывают в описании). Если содержание проблемы варьируется, все известные варианты указываются в описании.
- Когда? В какой момент работы программного продукта, по наступлению какого события или при каких условиях проблема проявляется.

В такой последовательности незнакомые дефекты удобнее сортировать по sumtagu (ведь, скорее всего, именно среди дефектов других инженеров будет производиться поиск дубликатов).

Серьезность. Если проблема найдена в ключевой функциональности приложения и после ее возникновения приложение становится полностью недоступно, и дальнейшая работа с ним невозможна, то она блокирующая. Обычно все блокирующие проблемы находятся во время первичной проверки новой версии продукта (Build Verification Test, Smoke Test), т.к. их наличие не позволяет полноценно проводить тестирование. Если же тестирование может быть продолжено, то серьезность данного дефекта будет критическая.

Шаги к воспроизведению / Результат / Ожидаемый результат. Важно четко описать все шаги, с упоминанием всех вводимых данных (имени пользователя, данных для заполнения формы) и промежуточных результатов. Пример такого описания показан на рис. 19.

Шаги к воспроизведению:

1. Войдите в системы: Пользователь Тестер1, пароль xxxXXX
--> Вход в систему осуществлен
2. Кликните линк «Профайл»
--> Страница «Профайл» открылась
3. Введите Новое имя пользователя: Тестер2
4. Нажмите кнопку «Сохранить»

Результат:

На экране появилась ошибка. Новое имя пользователя не было сохранено

Ожидаемый результат:

Страница профайл перегружилась. Новое значение имени пользователя сохранено.

Рис. 19. Описание дефекта

9.5. Основные ошибки при написании баг-репортов

1. Недостаточность предоставленных данных. Не всегда одна и та же проблема проявляется при всех вводимых значениях и под любым вошедшим в систему пользователем, поэтому настоятельно рекомендуется вносить все необходимые данные в баг-репорт.

2. Определение серьезности. Часто происходит либо завышение, либо занижение серьезности дефекта, что может привести к неправильной очередности при решении проблемы.

3. Язык описания. Часто при описании проблемы используются неправильная терминология или сложные речевые обороты, которые могут ввести в заблуждение человека, ответственного за решение проблемы.

4. Отсутствие ожидаемого результата. В случаях, если вы не указали, что же должно быть требуемым поведением системы, вы тратите время разработчика, на поиск данной информации, тем самым замедляете исправления дефекта. Вы должны указать пункт в требованиях, написанный тест-кейс или же ваше личное мнение, если эта ситуация не была документирована.

Задание

1. Выполнить поиск дефектов разработанного при выполнении лабораторной работы №7 программного продукта, указать их характеристики и в соответствии с предложенной выше структурой баг-репорт. Баг-репорт включается в отчет о выполнении лабораторной работы №10.

2. По согласованию с преподавателем рассмотреть сайты с целью поиска дефектов: <https://www.orange-elephant.ru/>, <https://flowwow.com/krasnodar/>, <https://sadvod-opt.com/> и др. Выбрать сайт некоторого интернет-магазина или аналогичного ресурса. Оформить соответствующий баг-репорт.

Контрольные вопросы и упражнения

1. Перечислите основные три цели тестирования программного обеспечения.
2. В чем отличия методик тестирования белого и черного ящиков?
3. Укажите уровни тестирования ПО.
4. Перечислите варианты серьезности дефекта.
5. Укажите обязательные поля баг-репортов.
6. Перечислите стадии жизненного цикла дефекта.

Лабораторная работа №10

Тема: Документирование и развертывание ПС.

Цель: Освоение методики документирования ПС.

Теоретические положения

10.1. Документирование ПО

Документирование относится к категории вспомогательных процессов жизненного цикла информационной системы, обеспечивающих выполнение основных процессов жизненного цикла ПС.

При осуществлении рабочего проектирования создаются следующие виды документации:

- функциональные спецификации – описывают функционирование отдельных компонентов ПС и системы в целом в отношении реализации функциональных требований к системе;
- проектные спецификации – раскрывают проектные решения, которые необходимы заказчику для выполнения функций сопровождения ПС;
- руководство пользователя – документ, регламентирующий действия пользователя при работе с ПС.

Документирование часто сопровождается диаграммами UML, которые позволяют визуально описывать приложение, наглядно представлять архитектуру и требования к приложению.

Основное требование к документации – она должна быть понятна и соответствовать функциональной и обеспечивающей структуре ПС. Основные ошибки при документировании ПС изложены в [подразделе 3.3](#) при рассмотрении задачи документирования требований. Это нечеткие формулировки, игнорирование аудитории, для которой предназначена данная документация, а также пропуск важных аспектов, связанных с нефункциональными требованиями.

10.2. Руководство пользователя

Руководство пользователя – это своего рода инструкция по эксплуатации приложения конечными пользователями. Предназначено для всех людей, принимающих участие в настройке и эксплуатации программного приложения.

Примерная структура руководства пользователя:

1. «Назначение приложения». Здесь задаются общие понятия, необходимые навыки для работы с системой:

- название приложения и номер версии;
- класс ПС, для которых предназначено приложение;

- степень универсальности приложения для решения поставленных задач;
- характеристики пользовательского интерфейса;
- уровень подготовки пользователей для эффективного взаимодействия с приложением;

2. «Установка приложения». В этой части описывается процесс установки и базовой настройки системы.

3. «Концепция приложения». Приводится общая структура системы, приводится справочник терминов и понятий, которые используются в интерфейсах управления системой и данном руководстве, описывается процесс функционирования в общем виде.

4. «Функциональный состав приложения». Рассматривается процесс решения задач на объекте в рамках функционального состава ПС. По ходу описания приводятся экранные формы всех интерфейсных окон / страниц, посредством которых пользователи взаимодействуют с приложением и устанавливаются ссылки на них.

5. «Инструменты и настройки приложения». Описывается назначение и применение настроек приложения, работа с модулями, обновлениями и полезными инструментами системы.

6. «Получение помощи». Приводятся способы решения возникших проблем и получения помощи (Help).

Задание

1. Собрать документированные материалы, разработанные при выполнении лабораторных работ №1–9 в единый документ с разбивкой на разделы. Темы лабораторных работ обозначить названиями разделов. В разделы не включать цель, задание и выводы из отчетов о выполнении лабораторных работ.
2. Дополнительно отдельным разделом 10 описать назначение, технические характеристики, принцип работы и меры безопасности при эксплуатации ПС.
3. Дополнительно отдельным разделом 11 составить руководство пользователя.

Контрольные вопросы и упражнения

1. Что такое документирование ПС и каково его назначение?
2. Какие элементы включает в себя структура документации к ПС?
3. Как документируются диаграммы в нотации IDEF0?
4. Что такое руководство пользователя?
5. Перечислите основные структурные элементы руководства пользователя.
6. Каково назначение руководства пользователя?
7. Какие задачи решаются на этапе эксплуатации ПС?

Список рекомендуемой литературы

Основная литература:

1. Полетайкин, А.Н. Методология и технология разработки программных систем: методы и модели программной инженерии: учебное пособие / А.Н. Полетайкин, Н.Ю. Добровольская; Министерство науки и высшего образования Российской Федерации, Кубанский государственный университет. – Краснодар: Кубанский гос. ун-т, 2025. – 229 с. Режим доступа: <http://212.192.134.46/MegaPro/Download/MObject/1389>.
2. Полетайкин, А.Н. Технологии разработки программных систем: учебное пособие / А.Н. Полетайкин, Н.Ю. Добровольская; Министерство науки и высшего образования Российской Федерации, Кубанский государственный университет. – Краснодар: Кубанский гос. ун-т, 2025. – 190 с.
3. Полетайкин А. Н. Социальные и экономические информационные системы: законы функционирования и принципы построения : Учеб. пособие / Сиб. гос. ун-т телекоммуникаций и информатики. - Новосибирск : СибГУТИ, 2016. - 241 с. Режим доступа: https://sibsutis.ru/upload/13e/up_is.pdf.

Дополнительная литература:

4. Гагарина Л.Г., Кокорева Е.В., Виснадул Б.Д. Технологии разработки программного обеспечения : учеб. пособие; под ред. Л.Г. Гагариной. – М.: ИД "ФОРУМ" : ИНФРА-М, 2012. – 399 с.
5. Штерн В. Основы C++. Методы программной инженерии: Пер. с англ. : монография. – М. : Лори, 2003. – 860с.
6. Мартин Р. Чистый код. Создание, анализ и рефакторинг : монография. – СПб.: ПИТЕР, 2012. – 464 с.
7. Мартишин С.А., Симонов В.Л., Храпченко М.В. Основы теории надёжности информационных систем : учеб. пособие. – М. : ФОРУМ: ИНФРА-М, 2013. – 254 с. : ил.
8. Орлов С.А. Технологии разработки программного обеспечения : Учебник для вузов. – СПб. : ПИТЕР, 2002. – 463 с.
9. Иванова Г.С. Технология программирования : учебник – 3-е изд., перераб. и доп. – М. : Изд-во МГТУ им. Н. Э. Баумана, 2006. – 335 с.
10. Липаев В.В. Технологии разработки программного обеспечения. Методологические основы: учебник. – М.: «ТЕИС», 2006 – 608 с.
11. Никитин В.А. Управление качеством на базе стандартов ИСО 9000:2000 : монография. – СПб. : ПИТЕР, 2002. – 262 с.
12. Губарев А.В. Информационное обеспечение системы менеджмента качества : моногр. – Москва : Горячая линия-Телеком, 2013. – 132 с. : ил.
13. Лодон Дж. Управление информационными системами : учебник. – 7-е изд. – СПб. : ПИТЕР, 2005. – 910 с.
14. Босуэлл Д., Фаучер Т. Читаемый код или Программирование как искусство : монография. – СПб. : ПИТЕР, 2012. – 203 с.

15. Зыков С.В. Основы современного программирования. Разработка гетерогенных систем в Интернет-ориентированной среде : учеб.пособие. – 2-е изд.,стер. – М. : Горячая линия-Телеком, 2012. – 444 с.

16. Кузнецов С.М., Малозёмов Б.В. Программирование и основы алгоритмизации : учеб. пособие. – Новосиб. гос. техн. ун-т. – Новосибирск : Изд-во НГТУ, 2006. – 199 с. : ил.

17. Кнут Д.Э. Искусство программирования: В 3 т. : Пер. с англ. Т.1:Основные алгоритмы : монография / Под общ. ред. Ю.В.Козаченко. – М. : Издат.дом "Вильямс", 2002. – 712 с.

ГОСТы:

18. ГОСТ Р ИСО/МЭК 12207–99. ГОСУДАРСТВЕННЫЙ СТАНДАРТ РОССИЙСКОЙ ФЕДЕРАЦИИ. Информационная технология. ПРОЦЕССЫ ЖИЗНЕННОГО ЦИКЛА ПРОГРАММНЫХ СРЕДСТВ.

19. ГОСТ Р ИСО/МЭК 15504-1-2009. Информационные технологии. Оценка процессов. Часть 1. Концепция и словарь Текст. Введ. 14.09.2009. – М. : Стандартиформ, 2010. – 19 с.

20. ГОСТ 34.601–90. Автоматизированные системы. Стадии создания. В кн.: Информационная технология. Комплекс стандартов и руководящих документов на автоматизированные системы. М.: Комитет стандартизации и метрологии СССР,1991. – с.45-52.

21. ГОСТ 34.602–89. Техническое задание на создание автоматизированной системы. В кн.: Информационная технология. Комплекс стандартов и руководящих документов на АС. М.: Комитет стандартизации и метрологии СССР,1991. – с.15-29

22. РД 50–34.698–90. Автоматизированные системы. Требования к содержанию документов. В кн.: Информационная технология. Комплекс стандартов и руководящих документов на АС. М.: Комитет стандартизации и метрологии СССР,1991. – с.66-104.

23. ГОСТ 19.701–90 (ИСО 5807-85). Схемы алгоритмов, программ, данных и систем. Госстандарт СССР, 1990. – 20 с.

Приложение А – Варианты индивидуальных заданий

1. Обменный пункт: сотрудники пункта, виды валют, курсы валют, операции обмена.
2. Ювелирный магазин: названия изделий, комитенты (кто сдал изделия на комиссию), журнал сдачи изделий на продажу, журнал покупки изделий.
3. Поликлиника: врачи, пациенты, виды болезней, журнал учета прихода пациентов.
4. Кондитерский магазин: виды конфет, поставщики, торговые точки, журнал поступления и отпуска товара.
5. Автобаза: автомашины, водители, рейсы, журнал выезда машин на рейсы.
6. Парикмахерская: клиенты, прайс услуг, сотрудники, кассовый журнал.
7. Школа: учителя, предметы, ученики, журнал успеваемости.
8. Оплата услуг на дачных участках: виды услуг, список владельцев, сотрудники управления, журнал регистрации оплат.
9. Гостиница: проживающие, сотрудники гостиницы, номера, журнал регистрации проживающих.
10. Книжный магазин: авторы, книги, продавцы, покупатели, регистрация поступлений и продаж.
11. Ремонтная мастерская: виды работ, исполнители, заказы на ремонт, заказчики.
12. Аптечный киоск: номенклатура лекарств, работники аптеки, покупатели, журнал регистрации продаж.
13. Выставка: стенды, стендисты, экскурсии, посетители.
14. Охранная служба: список постов охраны, список охранников, журнал выхода на дежурство, журнал учета замечаний.
15. Столовая: продукты, блюда, меню, журнал заказов
16. Фото мастерская: заказчики работ, прайс работ, журнал поступления заказов, исполнители.
17. Ветеринарная лечебница: список животных, список болезней, список хозяев, журнал посещений.
18. Сельское хозяйство: список растений, список угодий, список работников, журнал посевной.
19. Холдинг: список регионов, список предприятий, список показателей, журнал учета отчетных данных.
20. Фонды предприятия: список основных средств, список категорий основных средств, список материально ответственных лиц, журнал учета состояния основных средств.
21. Учет расхода материалов в компании: список статей затрат, список сотрудников, журнал учета расхода канцтоваров, список департаментов.
22. Фильмотека: список фильмов, список клиентов, список библиотекарей, журнал выдачи фильмов.
23. Цирк: список категорий артистов, список артистов, журнал выхода артистов на работу, список цирковых площадок.

24. Спортивные мероприятия: список спортсменов, список видов спорта, список стадионов, журнал учета выступлений спортсменов.
25. Компьютерные занятия: список слушателей курсов, список предметов, список преподавателей, журнал учета успеваемости.
26. Сбор урожая: список видов продукции, список сборщиков, список бригад, журнал учета сбора урожая.
27. Фирма по обслуживанию населения: список заказчиков, список товаров, список разносчиков, журнал заказов.
28. Партийная работа: список членов партии, список мероприятий, журнал учета выхода на мероприятие, список городов
29. Экономическая база данных: список регионов, список показателей, список отраслей, отчетные статистические данные.
30. Журнальные статьи: список тем, список авторов, список названия статей, список журналов.
31. Анализ причин заболеваемости: список больных, список болезней, список районов, журнал учета заболевших.
32. Отдел кадров: список сотрудников, штатное расписание, список отделов, журнал перемещения сотрудников по службе.
33. Расчет нагрузки на преподавателя: список преподавателей, список кафедр, предметов, журнал нагрузки.
34. Проектные работы: список проектов, список специалистов, список должностей, журнал учета работ.
35. Учет компьютерного оборудования: список типов оборудования, список материально ответственных лиц, список департаментов, журнал регистрации выдачи оборудования.
36. Прививки детям: список прививок, список детей, список родителей, журнал учета сделанных прививок.
37. Начисление налогов в бюджет: виды налогов, список отраслей, список предприятий, журнал учета поступления налогов.
38. Экспертная система: список оцениваемых объектов, список экспертов, список регионов, журнал учета оценок.
39. Ремонтная мастерская электронного оборудования: список работ, список мастеров, список запасных частей, журнал учета выполненных работ, список поступившего оборудования.
40. Магазин по продаже автомобилей: список фирм производителей, список автомобилей, журнал поступления автомобиля, список водителя пригнавшего машину.
41. Автомобильный гараж: список владельцев, список автомобилей, список сторожей, журнал прихода и ухода автомобилей.
42. Учет криминогенной ситуации в городе: список районов, список типов преступлений, список дежурных, журнал регистрации преступлений.
43. Система здравоохранения: список регионов, список санаториев, список пенсионеров, журнал регистрации выдачи путевок в санатории.
44. Туристические агентства: список туров, список стран, список клиентов, журнал регистрации продаж туров.

45. Продажа билетов на рейсы: список рейсов, прайс билетов, список компаний, журнал продаж билетов.
46. Продажа пиломатериалов: виды пиломатериалов, регионы поставщики, список заказчиков, журнал учета продаж пиломатериалов.
47. Склад металлоконструкций: прайс товара металлоконструкций, список поставщиков, список продавцов, журнал учета продаж.
48. Система поддержки решений: список экспертов, список тем обсуждений, список департаментов, журнал учета предложений.
49. Детский сад: список родителей, список детей, список групп, журнал посещения детского сада.
50. Дом творчества молодежи: список кружков, список руководителей, список детей, журнал регистрации посещения кружков.

Приложение Б – Примерное описание объекта информатизации

Характеристика объекта информатизации

Объектом информатизации является ПАО Банк «ФК Открытие». Банк «Открытие» – крупнейший частный банк в России и четвертый по размеру активов среди всех российских банковских групп. Работает на финансовом рынке с 1993 года [1].

Активы банка и его дочерних компаний по международным стандартам финансовой отчетности (МСФО) на 30 июня 2016 года составили 3 087,8 млрд рублей, собственный капитал – 217,8 млрд рублей.

«Открытие» – универсальный коммерческий банк с устойчивой диверсифицированной структурой бизнеса и качественным управлением капиталом.

Банк развивает следующие ключевые направления бизнеса:

- корпоративный;
- инвестиционный;
- розничный;
- малый и средний бизнес;
- приват банкинг.

Особое внимание «Открытие» уделяет высокотехнологичным сервисам. Так, в рамках проекта «Рокетбанк» предлагается полностью дистанционный сервис для физических лиц, а в рамках проекта «Точка» – полностью дистанционное обслуживание для предпринимателей.

Банк «Открытие» был сформирован в результате интеграции более чем десяти банков различного масштаба, в том числе таких крупных федеральных, как НОМОС-БАНК, Ханты-Мансийский банк и банк «Петрокоммерц».

Банк «Открытие» входит в список десяти системообразующих кредитных организаций, утвержденный Центральным банком, а также в список крупнейших компаний мира FORBES Global 2000.

Надежность банка подтверждена рейтингами международных агентств S&P Global (BB-) и Moody's Investors Service (Ba3).

Клиентская база объединенного банка насчитывает свыше 30 000 корпоративных клиентов, 165 000 клиентов малого бизнеса и около 3,2 млн физических лиц, в том числе премиальных клиентов. 470 отделений банка различного формата расположены в 53 экономически значимых регионах России. Значительная часть бизнеса сосредоточена в Москве, Санкт-Петербурге, Тюменской области, Екатеринбурге, Новосибирской области, Хабаровском крае, Волгоградской области.

Ключевым акционером банка «Открытие» является «Открытие Холдинг», который владеет 64,7% голосующих акций (см. рис.). Бенефициарами «Открытие Холдинг» являются: Вадим Беляев, Группа «ИФД Капиталь», Банк «ВТБ»,

Группа «ИСТ», Рубен Аганбегян, Александр Мамут. Бумаги банка также находятся в свободном обращении на Московской бирже.



Рисунок – Структура ОАО «Открытие Холдинг»

Главой Наблюдательного совета банка «Открытие» является Дмитрий Ромаев, председателем Правления – Евгений Данкевич.

Банк «Открытие» активно участвует в реализации социально значимых проектов, поддерживая такие направления, как наука (Политехнический музей), образование (Московский государственный университет им. Ломоносова, Европейский университет в Санкт-Петербурге), культура (Малый театр, сотрудничество с автором мультфильма «Ёжик в тумане» Юрием Норштейном), здравоохранение (Фонд помощи хосписам «Вера»), спорт (стратегическое партнерство с ФК «Спартак-Москва», Югорский лыжный марафон).

Для информатизации было выбрано отделение банка в г. Новосибирске, располагающееся по адресу: 630102, г. Новосибирск, ул. Кирова, д. 44.

Контактный телефон: (383) 325-10-85.

Управляющий филиалом: Ермилов Валерий Николаевич.

Главный бухгалтер: Жданов Алексей Владимирович.

Филиал обслуживает как частные, так и юридические лица.

Услуги частным лицам включают в себя:

- срочные переводы по системе «Western Union» в рублях (по России) и в валюте в любую точку мира;
- прием коммунальных платежей;
- оплата услуг операторов мобильной связи;
- оплата кредитов, полученных в других банках;
- автокредитование;
- кредиты по пластиковым картам;
- открытие и ведение текущих рублёвых и валютных счетов;
- предоставление индивидуальных банковских сейфов;

- оформление и обслуживание пластиковых карт;
- денежные вклады;
- ипотечное кредитование;
- инвестиции;
- страхование;
- пенсионный калькулятор;
- сейфовые ячейки;
- операции с иностранной валютой и чеками.

Услуги корпоративным клиентам включают в себя:

- расчётно-кассовое обслуживание, в том числе по системе «Банк-Клиент»;
- широкий выбор кредитных продуктов (кредитные линии, овердрафт, вексельные кредиты, лизинг);
- оперативное проведение валютно-финансовых операций;
- вклады и депозиты для корпоративных клиентов;
- организация процедуры выплаты зарплаты с помощью пластиковых карт;
- оформление пластиковых корпоративных карт международных платёжных систем;
- консалтинговые услуги;
- дистанционное обслуживание;
- инвестиции.

Процессами информатизации являются: расчёт ежемесячного платежа для физических лиц и оценка их кредитоспособности при ипотечном кредитовании.

Ипотечное кредитование – долгосрочный кредит, предоставляемый юридическому или физическому лицу банками под залог недвижимости (земли, производственных и жилых зданий, помещений, сооружений). Самый распространенный вариант использования ипотеки в России – это покупка физическим лицом квартиры в кредит. Закладывается при этом, как правило, вновь покупаемое жилье, хотя можно заложить и уже имеющуюся в собственности квартиру [2].

Цель информатизируемых процессов: рассчитать ежемесячный платёж и проверить кредитоспособность заёмщика.

Ограничения на операции: возраст клиента должен быть не менее 18 лет и не более 65 лет на момент выплаты последнего платежа.

Перечень ссылок

1 Банк «Финансовая корпорация Открытие» [Офиц. сайт]. URL: <https://www.open.ru> (дата обращения: 17.05.2017).

2 IPOhelp [Офиц. сайт]. URL: <http://www.ipohelp.ru/manual.html> (дата обращения: 10.02.2017).

Приложение В – Показатели для оценивания программной системы
(согласно ГОСТ Р ИСО/МЭК 9126-93)

1. Целостность – способность системы противостоять (сохранять информационное содержание и однозначность интерпретации смысла) изменениям, искажениям или порче при возникновении сбоев или ошибок.
2. Рациональность – подразумевает, что при функционировании оптимальным образом используются имеющиеся в распоряжении системы ресурсы: время, оборудование (память), люди.
3. Численность задействованного персонала – количество лиц, задействованных в эксплуатации и обслуживании системы.
4. Работоспособность – способность системы выполнять свои функции с эксплуатационными показателями не ниже заданных.
5. Производительность – характеристика системы, отражающая ее способность производить определенный объем работ в единицу времени, например, пропускная способность, время ответа, доступность, число продуктов, полученная прибыль, быстродействие.
6. Гибкость – Возможность модификации обеспечивающей части системы, обычно возникает по двум причинам: чтобы отразить в системе изменение требований или чтобы исправить ошибки, внесённые ранее в процессе разработки. Гибкость заключается в возможности адаптации, наращивания, изменения средств.
7. Открытость – прозрачность функциональной части системы и возможность ее модификации без нарушения процесса функционирования.
8. Защищенность (Безопасность) – способность обеспечения защиты данных от разрушения, искажения или преднамеренных фальсификаций злоумышленником. Характеризует возможное отсутствие риска, связанного с нанесением некоторого ущерба
9. Потенциальная управляемость – возможность компенсировать возмущение быстрее, чем успеют измениться эти возмущения.
10. Наблюдаемость основных параметров управляемого процесса – характеризует степень и глубину отслеживания параметров функционирования системы и основных характеристик объекта управления на всех циклах и этапах работы системы/объекта.
11. Надёжность – свойство системы сохранять во времени в установленных пределах значения всех параметров, характеризующих способность системы выполнять требуемые функции в заданных режимах и эксплуатации.
12. Степень оперативности и надежности управления – характеризует способность системы и субъекта управления выполнять поставленные задачи в сроки, обеспечивающие эффективное управление объектом. Управление является оперативным, если время, затрачиваемое на решение системой задач, в совокупности со временем, необходимым персоналу на подготовку к реализации управленческих решений, не будет превышать критического времени, диктуемого условиями сложившейся ситуации на объекте. О.у.

достигается ускорением процесса сбора и обработки данных о состоянии объекта, принятия решений, планирования прогнозирования социально-экономических показателей.

- 13.Безотказность – свойство системы сохранять работоспособность в течение требуемого интервала времени непрерывно без вынужденных перерывов. Безотказность является наиболее важной компонентой надёжности, так как она отражает способность длительное время функционировать без отказов. Один из показателей надёжности системы.
- 14.Живучесть – свойство системы сохранять работоспособность в условиях возмущающих воздействий внешней среды и отказов компонентов системы с минимальной частотой отказов, а в случае их возникновения эффективно восстанавливать утраченные функции и ресурсы.
- 15.Прогрессивность компьютерных и информационных технологий, задействованных в системе
- 16.Наглядность интерфейса – должен быть удобным, интуитивно понятным и продуманным с точки зрения инженерной психологии, эргономики и методов технической эстетики
- 17.Долговечность — свойство системы сохранять работоспособность до наступления предельного состояния. Зависит от долговечности технических средств и подверженности системы моральному старению.
- 18.Сопровождаемость – отражает возможность проведения конкретных изменений (модификаций) системы. Изменение может включать исправления, усовершенствования или адаптацию компонентов системы к изменениям в окружающей обстановке, требованиях и условиях функционирования.
- 19.Стоимость – овеществленный в товаре труд разработчиков. Определяется количеством труда, материала, энергии и информации, затраченных на производство товаров, и измеряется количеством, например эквивалента рабочего времени.
- 20.Эффективность – получение от функционирования системы существенного технико-экономического, социального или другого эффекта.
- 21.Мобильность – возможность перенесения системы из одного окружения в другое. Окружающая обстановка может включать организационное, техническое или программное окружение, а также условия эксплуатации.
- 22.Наличие тех. поддержки. Введенная в эксплуатацию готовая система требует определенной технической поддержки. Это обусловлено, прежде всего, динамичностью информационных процессов: совершенствованием документооборота, появлением дополнительных структур данных и автоматизированных функций, что является обычным явлением для любых развивающихся систем.

**Приложение Г – Состав и содержание технического задания на создание программ
(ГОСТ 34.602- 89)**

№ п/п	Раздел	Содержание
1	Общие сведения	• полное наименование программы и ее условное обозначение • шифр темы или шифр (номер) договора; • наименование предприятий разработчика и заказчика системы, их реквизиты • перечень документов, на основании которых создается программа • плановые сроки начала и окончания работ • сведения об источниках и порядке финансирования работ • порядок оформления и предъявления заказчику результатов работ по созданию системы, ее частей и отдельных средств
2	Назначение и цели создания (развития) программы	• вид автоматизируемой деятельности • перечень объектов, на которых предполагается использование программы • наименования и требуемые значения технических, технологических, производственно-экономических и др. показателей объекта, которые должны быть достигнуты при внедрении программы
3	Характеристика бизнес-процесса	• краткие сведения об объекте автоматизации • сведения об условиях эксплуатации и характеристиках окружающей среды
4	Состав и содержание работ по созданию программы	• перечень стадий и этапов работ • сроки исполнения • состав организаций — исполнителей работ • вид и порядок экспертизы технической документации • программа обеспечения надежности • программа математического обеспечения
5	Порядок контроля и приемки программы	• виды, состав, объем и методы испытаний программы • общие требования к приемке работ по стадиям • статус приемной комиссии
6	Требования к составу и содержанию работ по подготовке объекта автоматизации к вводу программы в действие	• преобразование входной информации к машиночитаемому виду • изменения в объекте автоматизации • сроки и порядок комплектования и обучения персонала
7	Требования к документированию	• перечень подлежащих разработке документов • перечень документов на машинных носителях
8	Источники разработки	• документы и информационные материалы, на основании которых разрабатывается ТЗ и программы

Приложение Д – Дополнительное задание на проведение сценарного тестирования

В качестве тестируемого программного продукта может быть выбран некоторый интернет-магазин, например:

- <https://www.orange-elephant.ru/>;
- <https://flowwow.com/krasnodar/>;
- <https://sadovod-opt.com/> и др.).

Для разработки сценария необходимо выбрать три задачи из списка ниже. Например, в соответствии с задачей 8 из списка ниже один из сценариев может быть следующим: *поиск горнолыжной мужской куртки 52 размера, с ограничением стоимости.*

Перечень задач для сценарного тестирования.

1. Ориентирование в категориях каталога. Вариант 1
2. Ориентирование в категориях каталога. Вариант 2
3. Фильтрация по заданным параметрам
4. Сброс фильтров
5. Сортировка по заданному параметру
6. Изучение карточки товара в каталоге
7. Изучение страницы с подробным описанием товара
8. Поиск товара по заданным параметрам
9. Сравнение товаров
10. Ориентирование в подборках товаров
11. Добавление товара в избранное
12. Работа с изображением товара
13. Ориентирование в стоимости товара
14. Взаимодействие с чужими отзывами
15. Считываемость доступного кол-ва товара
16. Обращение в поддержку
17. Заказ в один клик
18. Добавление товаров в корзину
19. Удаление товара из корзины
20. Очистка корзины
21. Применение промокодов
22. Поиск ближайшей точки по карте
23. Понимание условий возврата товаров
24. Понимание условий программы лояльности
25. Понимание Push/SMS о статусе заказа
26. Считываемость баннеров
27. Перелистывание карусели
28. Отмена заказа
29. Трекинг заказа
30. Ориентирование в стоимости опций и заказа
31. Считываемость рекомендуемых товаров
32. Поиск заданной точки по карте

33. Оформление доставки
34. Смена города/региона
35. Работа с кликабельными тэгами
36. Оставление фидбека о товаре
37. Взаимодействие с видео о товаре
38. Передача ссылок и шеринг информации о товаре
39. Использование списка избранного
40. Повторение прошлых заказов
41. Редактирование профиля
42. Редактирование состава заказа в корзине
43. Изменений опций заказа
44. Выбор способа оплаты
45. Выбор способа получения товара
46. Считываемость экранов/сообщений об ошибке
47. Взаимодействие с всплывающими подсказками
48. Регистрация
49. Поиск товаров, похожих на просматриваемый
50. Считываемость продвигаемых товаров
51. Оформление самовывоза
52. Свободный поиск товара
53. Работа с поисковой строкой
54. Оплата заказа
55. Поиск товара через каталог